LangChain & LangGraph

The Complete Guide — from first chain to production multi-agent systems

Covers LangChain v1 (create_agent, middleware) and LangGraph v1, plus the legacy APIs you will still meet in older code and assessments.

Inside: 25 chapters • Runnables & LCEL in depth • every composition primitive • prompts, parsers, structured output • chains (legacy → modern) • memory • loaders, splitters, vector stores, retrievers • tools & tool calling • agents & middleware • callbacks & LangSmith • LangGraph state, reducers, nodes, edges, Send, Command • workflow patterns • ReAct & agentic RAG graphs • checkpointers, time travel • short & long-term memory • human-in-the-loop • streaming • subgraphs & multi-agent • Functional API • deployment & testing • 2 end-to-end projects • ~90 interview Q&A • cheat sheets

Prepared for Mansavi Aggarwal

Contents

1. Introduction: What LangChain & LangGraph Are **7**

1.1 Why do we need LLM frameworks? . . . 7

1.2 The ecosystem and packages . . . 7

1.3 Installation & setup . . . 9

1.4 LangChain vs LangGraph — when to use which . . . 9

2. Chat Models & Messages **11**

2.1 Chat models . . . 11

2.2 Messages . . . 12

2.3 Rate limiting, caching and fallbacks . . . 13

2.4 Embedding models . . . 13

3. Prompt Templates **14**

3.1 PromptTemplate and ChatPromptTemplate . . . 14

3.2 Few-shot prompting . . . 15

3.3 Prompt management . . . 15

4. Output Parsers & Structured Output **16**

4.1 Why parse outputs? . . . 16

4.2 with_structured_output (preferred) . . . 16

4.3 Output parsers . . . 17

5. Runnables & LCEL (LangChain Expression Language) **18**

5.1 The Runnable protocol — the heart of LangChain . . . 18

5.2 LCEL composition primitives . . . 19

5.3 RunnableConfig — controlling execution . . . 21

5.4 Wrapping runnables: bind, retry, fallbacks, configurable . . . 21

5.5 Streaming deep dive . . . 22

5.6 Inspecting and visualising chains . . . 22

6. Chains: Legacy and Modern **23**

6.1 Chains, then and now . . . 23

6.2 Document chains: combining many documents . . . 24

6.3 Extraction and tagging chains . . . 25

7. Memory & Chat History **26**

7.1 LLMs are stateless . . . 26

7.2 Legacy memory classes (langchain-classic) — recognise them . . . 26

7.3 Chat message histories and RunnableWithMessageHistory . . . 27

8. Documents, Loaders, Splitters, Vector Stores & Retrievers (RAG in LangChain) 28

8.1 The Document object . . . 28
8.2 Document loaders . . . 28
8.3 Text splitters . . . 29
8.4 Vector stores . . . 29
8.5 Retrievers . . . 30
8.6 Indexing API (keeping the vector store in sync) . . . 31
8.7 Putting it together: a complete RAG chain . . . 31

9. Tools & Tool Calling 33

9.1 What is a tool? . . . 33
9.2 Creating tools . . . 33
9.3 Binding tools to a model and executing calls . . . 34

10. Agents in LangChain 35

10.1 What is an agent? . . . 35
10.2 Legacy agents (recognise in old code) . . . 35
10.3 create_agent — the v1 standard . . . 36
10.4 Middleware — customising the agent loop . . . 37

11. Callbacks, LangSmith, Debugging & Evaluation 39

11.1 Callbacks . . . 39
11.2 LangSmith: tracing and evaluation . . . 39
11.3 Debugging and testing . . . 40

12. Production Patterns for LangChain Apps 41

12.1 Serving with FastAPI . . . 41
12.2 Production checklist . . . 41
12.3 MCP tools . . . 42

13. LangGraph Fundamentals: Mental Model 44

13.1 Why LangGraph? . . . 44
13.2 How a graph executes (Pregel model) . . . 44
13.3 Your first graph . . . 45
13.4 Graph API vs Functional API . . . 46

14. State, Nodes and Edges in Depth 47

14.1 State schemas . . . 47

14.2 Reducers . . . 47
14.3 Nodes . . . 48
14.4 Edges . . . 49
14.5 Recursion limit and loops . . . 50

15. Workflow Patterns & Building Agents in LangGraph 51

15.1 Six core patterns . . . 51
15.2 A ReAct agent from scratch . . . 52
15.3 Agentic RAG as a graph (CRAG + Self-RAG style) . . . 53

16. Persistence, Threads, Checkpoints & Time Travel 54

16.1 Checkpointers and threads . . . 54
16.2 Inspecting and modifying state . . . 55
16.3 Time travel: replay and fork . . . 55
16.4 Durability and fault tolerance . . . 56

17. Memory: Short-Term and Long-Term 57

17.1 Short-term memory . . . 57
17.2 Long-term memory with the Store . . . 58

18. Human-in-the-Loop 59

18.1 Dynamic interrupts . . . 59
18.2 Common HITL patterns . . . 59
18.3 Rules you must know (frequent interview / debugging topics) . . . 60

19. Streaming 61

20. Subgraphs & Multi-Agent Systems 62

20.1 Subgraphs . . . 62
20.2 Multi-agent architectures . . . 62

21. Functional API 65

22. Deployment, Testing & Production 66

22.1 Deployment options . . . 66
22.2 Testing LangGraph applications . . . 67
22.3 Production checklist . . . 67

23. End-to-End Projects 69

23.1 Project 1 — Customer support agent with create_agent . . . 69
23.2 Project 2 — Research assistant with LangGraph . . . 70

24. Interview Questions & Answers **71**

25. Cheat Sheets **78**

25.1 Import cheat sheet (v1) 78

25.2 Skeletons to memorise 79

25.3 Common errors and fixes 79

25.4 Legacy → modern mapping 80

Sources 80

Part I — LangChain

Models, messages, prompts, parsers, Runnables & LCEL, chains, memory, RAG components, tools, agents & middleware, callbacks, LangSmith, production

CHAPTER 1

Introduction: What LangChain & LangGraph Are

Why frameworks • Ecosystem & packages • v1 changes • Installation & setup • When to use LangChain vs LangGraph • Learning map

1.1 Why do we need LLM frameworks?

Calling an LLM API once is easy. Real applications need much more: prompt templates, conversation history, output parsing into Python objects, tool calling, retrieval from documents, retries and fallbacks, streaming, switching providers, tracing and evaluation, and multi-step control flow. **LangChain** gives standard, provider-agnostic building blocks for these, and **LangGraph** gives a low-level runtime to orchestrate them as stateful, durable graphs (agents and workflows).

Problem without a framework	What LangChain / LangGraph provide
Every provider (OpenAI, Gemini, Anthropic…) has a different SDK and message format	one ChatModel interface + standard message types; swap providers by changing one string
Prompts as messy f-strings	PromptTemplate / ChatPromptTemplate with variables, few-shot examples and message placeholders
Free-text answers you must parse	output parsers and with_structured_output(PydanticModel)
Hand-written tool-calling loops	@tool, bind_tools, create_agent, ToolNode
Gluing steps together	the Runnable interface + LCEL (| operator) with built-in batch, stream and async
RAG plumbing	loaders, splitters, embeddings, vector stores, retrievers
Loops, branching, memory, human approval, crash recovery	LangGraph: StateGraph, conditional edges, checkpointers, interrupts
Debugging what happened	callbacks + LangSmith tracing and evaluation

1.2 The ecosystem and packages

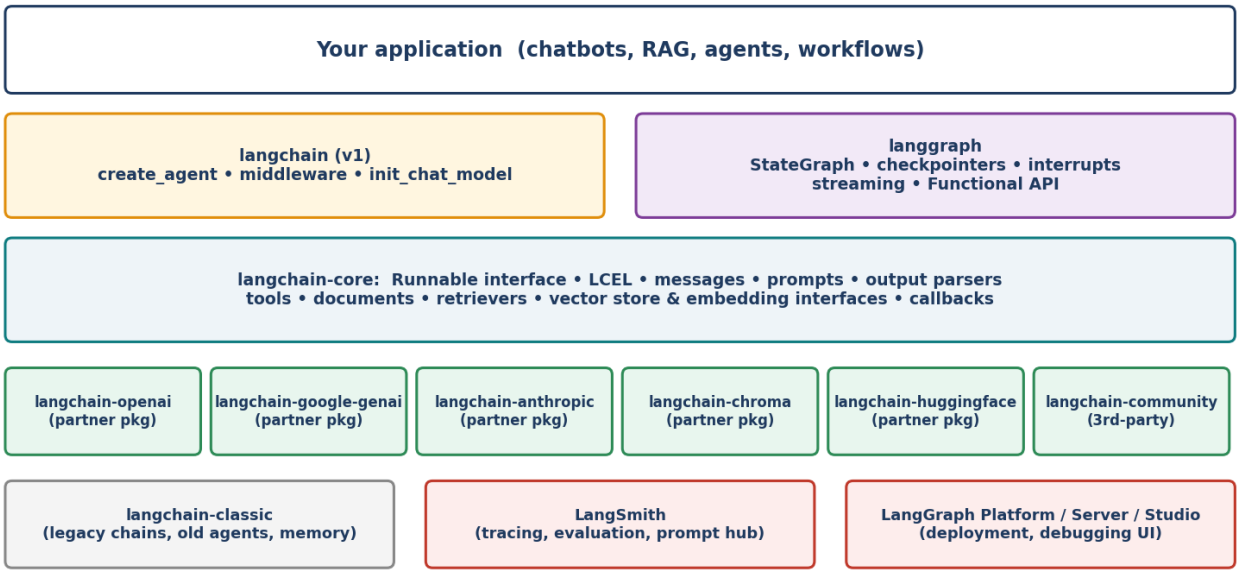


Figure: How the LangChain packages fit together.

Package	Contains	Install
langchain-core	base abstractions: Runnable, LCEL, messages, prompts, parsers, tools, documents, retrievers, callbacks	installed automatically
langchain	high-level: create_agent, middleware, init_chat_model, init_embeddings; re-exports core pieces	pip install -U langchain
Partner packages	provider integrations maintained jointly: langchain-openai, langchain-google-genai, langchain-anthropic, langchain-ollama, langchain-chroma, langchain-huggingface, langchain-qdrant…	pip install langchain-google-genai
langchain-community	community integrations (many loaders, vector stores, tools)	pip install langchain-community
langchain-text-splitters	text splitters	pip install langchain-text-splitters
langchain-classic	legacy chains (LLMChain, RetrievalQA…), old agents (AgentExecutor), old memory classes, indexing helpers moved out of langchain v1	pip install langchain-classic
langgraph	StateGraph, Functional API, checkpointers, interrupts, Send/Command, prebuilt ToolNode	pip install -U langgraph
langgraph-checkpoint-sqlite / -postgres	persistent checkpointers	pip install langgraph-checkpoint-sqlite
langsmith	tracing, datasets, evaluation SDK	pip install langsmith
langchain-mcp-adapters	load tools from MCP servers	pip install langchain-mcp-adapters

What changed in v1 (October 2025) — important for interviews

LangChain 1.0 and LangGraph 1.0 were released together, with a promise of no breaking changes until 2.0.[1] In LangChain v1:

- **create_agent** (from langchain.agents) is the standard way to build agents, replacing langgraph.prebuilt.create_react_agent; it runs on LangGraph under the hood.[2]
- **Middleware** is the key customisation mechanism of create_agent — hooks around model calls, tool calls and the agent loop for dynamic prompts, summarisation, tool selection, guardrails and human approval.[2]

- **Standard content blocks**: a content_blocks property gives unified access to provider features (reasoning, citations, images) across providers.[2]
- **Simplified namespace**: legacy chains/agents/memory moved to **langchain-classic**.[2]
- LangGraph v1 kept the graph APIs (state, nodes, edges) and runtime unchanged; it is a stability release, and its create_react_agent prebuilt is deprecated in favour of create_agent.[3]

> **Watch out:** You will still see AgentExecutor, LLMChain, ConversationBufferMemory and RetrievalQA in older tutorials, assessments and codebases. This book teaches both: the **legacy** API (so you can read old code) and the **modern** equivalent (so you write new code correctly).

1.3 Installation & setup

setup

```
python -m venv .venv && .venv\Scripts\activate          # Windows (Linux/Mac: source .venv/bin/activate)
pip install -U langchain langgraph langchain-google-genai langchain-openai \
               langchain-chroma langchain-huggingface langchain-community langchain-text-splitters \
               langgraph-checkpoint-sqlite python-dotenv
# .env  (never commit this file)
GOOGLE_API_KEY=...            # Gemini
OPENAI_API_KEY=...            # OpenAI
LANGSMITH_TRACING=true        # optional tracing
LANGSMITH_API_KEY=...
LANGSMITH_PROJECT=my-project
```

hello_langchain.py

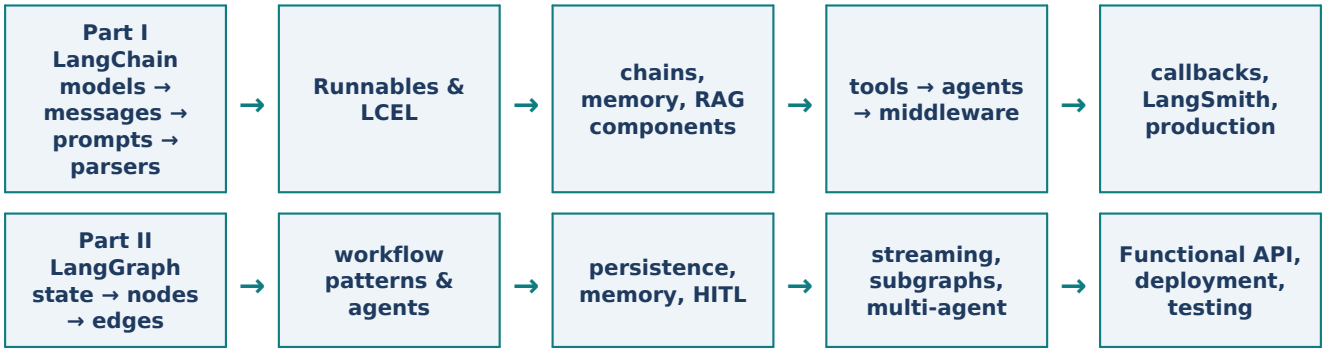
```
from dotenv import load_dotenv; load_dotenv()
from langchain.chat_models import init_chat_model
llm = init_chat_model("google_genai:gemini-2.5-flash", temperature=0)
print(llm.invoke("Say hello in one word").content)
```

1.4 LangChain vs LangGraph — when to use which

	LangChain	LangGraph
Level	high-level components + prebuilt agent	low-level orchestration runtime
Control flow	linear chains (LCEL) or the standard agent loop	any graph: loops, branches, parallel, sub-graphs
State	messages (agent state extendable)	custom typed state with reducers
Persistence / HITL	via create_agent (uses LangGraph checkpointer + middleware)	first-class: checkpointers, interrupts, time travel
Best for	fast start: RAG chains, extraction, standard tool agents	custom, long-running, multi-agent, production-grade workflows

LangChain's own guidance: use LangChain to get started quickly with a standard agent, and drop down to LangGraph when you need custom orchestration; they are designed to work together.[1][3] Many real projects use LangChain *components* (models, prompts, tools, retrievers) *inside* LangGraph *nodes*.

Learning map of this book



CHAPTER 2

Chat Models & Messages

Chat models vs LLMs • init_chat_model • parameters • invoke / stream / batch • messages & roles • content blocks • token usage • multimodal • rate limiting & caching • embeddings

2.1 Chat models

A **chat model** takes a list of **messages** and returns an **AIMessage**. Older 'LLM' classes took a string and returned a string — today nearly everything uses chat models. Every chat model is a **Runnable**, so it supports invoke, stream, batch and their async versions.

chat_models.py

```
# Option 1: provider class
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_openai import ChatOpenAI
gemini = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0, max_tokens=None, timeout=30,
        max_retries=2)
gpt = ChatOpenAI(model="gpt-4o-mini", temperature=0.2)

# Option 2: provider-agnostic factory (recommended in v1)
from langchain.chat_models import init_chat_model
llm = init_chat_model("google_genai:gemini-2.5-flash", temperature=0)
llm2 = init_chat_model("gpt-4o-mini", model_provider="openai")
# configurable model: choose provider at runtime
flex = init_chat_model(temperature=0, configurable_fields=("model", "model_provider"))
flex.invoke("hi", config={"configurable": {"model": "gemini-2.5-flash", "model_provider": "google_genai"}})
```

Parameter	Meaning
model	model ID (check provider docs; IDs change)
temperature	randomness: 0 for factual/extraction/RAG, 0.7+ for creative
max_tokens	cap on output length/cost
timeout, max_retries	network robustness
top_p, stop	nucleus sampling, stop sequences
api_key / base_url	normally from env vars; base_url for proxies/compatible servers (vLLM, Ollama)
rate_limiter	InMemoryRateLimiter to stay under quotas

Invocation methods

invoke_stream_batch.py

```
r = llm.invoke("Explain RAG in one line")              # AIMessage
print(r.content, r.usage_metadata, r.response_metadata)  # text, token counts, finish reason/model

for chunk in llm.stream("Write a haiku about Bengaluru"):  # AIMessageChunk objects
    print(chunk.content, end="", flush=True)

answers = llm.batch(["Define LLM", "Define RAG"], config={"max_concurrency": 5})   # parallel
r = await llm.ainvoke("async call")                       # inside async def
async for c in llm.astream("..."): ...
```

Chunks can be added: *full = chunk1 + chunk2 + …* builds the complete message (including tool-call fragments). **usage_metadata** = {'input_tokens', 'output_tokens', 'total_tokens'} — log it for cost tracking.

2.2 Messages

Message class	Role	Used for
SystemMessage	system	instructions, persona, rules — usually first
HumanMessage	user	user input (text, images, files)
AIMessage	assistant	model output: content, tool_calls, usage_metadata, response_metadata, id
ToolMessage	tool	result of executing a tool; must carry tool_call_id matching the request
AIMessageChunk	assistant	streamed piece of an AIMessage
RemoveMessage	—	special marker used in LangGraph to delete a message by id

messages.py

```
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage, ToolMessage
msgs = [SystemMessage("You are a concise banking assistant."),
        HumanMessage("What is a fixed deposit?"),
        AIMessage("A fixed deposit is a savings product with a fixed rate and tenure."),
        HumanMessage("And the typical lock-in?")]
llm.invoke(msgs)
# Equivalent shorthand formats accepted everywhere:
llm.invoke([("system", "You are concise."), ("human", "Hi")])
llm.invoke([{"role": "user", "content": "Hi"}])
```

Content and content blocks

message.content is either a string or a list of typed blocks (text, image, file, reasoning, tool call…). In v1, **message.content_blocks** gives a standardised list across providers — useful for reasoning traces, citations and multimodal outputs.[2]

multimodal.py

```
msg = HumanMessage(content=[
    {"type": "text", "text": "What is in this invoice? Return the total."},
    {"type": "image", "url": "https://example.com/invoice.png"},          # or base64: {"type":"image","base64":...,
        "mime_type":"image/png"}
])
resp = llm.invoke([msg])
for block in resp.content_blocks: print(block["type"])
```

Message utilities

Utility	Purpose
trim_messages(msgs, max_tokens=..., strategy='last', token_counter=llm, start_on='human', include_system=True)	keep the conversation within the context window
filter_messages(msgs, include_types=[...])	select messages by type/name/id
merge_message_runs(msgs)	merge consecutive same-role messages (some providers require alternation)
message.pretty_print()	readable debug output
convert_to_messages / messages_to_dict	serialisation

2.3 Rate limiting, caching and fallbacks

rate_cache_fallback.py

```
from langchain_core.rate_limiters import InMemoryRateLimiter
limiter = InMemoryRateLimiter(requests_per_second=2, check_every_n_seconds=0.1, max_bucket_size=5)
llm = init_chat_model("google_genai:gemini-2.5-flash", rate_limiter=limiter)

from langchain_core.globals import set_llm_cache
from langchain_core.caches import InMemoryCache            # SQLiteCache in langchain_community.cache
set_llm_cache(InMemoryCache())                             # identical prompt → cached response, no API cost

robust = llm.with_retry(stop_after_attempt=3).with_fallbacks([init_chat_model("openai:gpt-4o-mini")])
```

2.4 Embedding models

Embedding models turn text into vectors for semantic search (RAG). Interface: **embed_documents(list[str]) → list[list[float]]** and **embed_query(str) → list[float]**.

embeddings.py

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_google_genai import GoogleGenerativeAIEmbeddings
emb = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")      # local, 384-d
# emb = GoogleGenerativeAIEmbeddings(model="models/gemini-embedding-001")             # API (check current model
        id)
vecs = emb.embed_documents(["refund policy", "leave policy"]); q = emb.embed_query("how to get a refund")
print(len(vecs[0]))
from langchain.embeddings import init_embeddings                                      # v1 factory
emb2 = init_embeddings("openai:text-embedding-3-small")
```

CHAPTER 3

Prompt Templates

PromptTemplate • ChatPromptTemplate • MessagesPlaceholder • partial variables • few-shot prompting • example selectors • prompt composition • LangSmith prompt hub

3.1 PromptTemplate and ChatPromptTemplate

A prompt template is a **Runnable** that takes a dict of variables and returns a **PromptValue** (a string for LLMs or a list of messages for chat models). Templates separate fixed instructions from variable data, make prompts testable and versionable, and plug directly into LCEL chains.

prompts.py

```
from langchain_core.prompts import PromptTemplate, ChatPromptTemplate, MessagesPlaceholder
p = PromptTemplate.from_template("Summarise in {n} bullet points:\n{text}")
p.invoke({"n": 3, "text": "..."})                      # StringPromptValue
p.input_variables                                       # ['n', 'text']

chat = ChatPromptTemplate.from_messages([
    ("system", "You are a {role}. Answer in {language}."),
    MessagesPlaceholder("history", optional=True),      # inject a list of past messages
    ("human", "{question}"),
])
pv = chat.invoke({"role": "HR assistant", "language": "English", "history": [], "question": "Leave policy?"})
pv.to_messages()                                        # [SystemMessage, HumanMessage]
```

Feature	How	Why
Variables	{name} placeholders (f-string format by default; template_format='jinja2' or 'mustache' available)	reuse
Escaping braces	{{ and }} for literal braces (important when the prompt contains JSON examples!)	avoid KeyError
MessagesPlaceholder	insert a variable-length list of messages (chat history, agent scratchpad)	memory
partial()	pre-fill some variables: chat.partial(language='English') or with a function (e.g. today's date)	defaults
Composition	prompt1 + prompt2, or ChatPromptTemplate with multiple message templates	modularity
Image inputs	message templates with image_url/image blocks and variables	multimodal

Watch out: Common error: *KeyError / missing variables* when a prompt contains literal JSON like {"label": "x"}. Escape as {{"label": "x"}} or pass the JSON via a variable.

3.2 Few-shot prompting

few_shot.py

```
from langchain_core.prompts import FewShotChatMessagePromptTemplate
examples = [{"ticket": "Card blocked abroad", "label": "cards"}, {"ticket": "Charged twice for EMI", "label": "billing"}]
example_prompt = ChatPromptTemplate.from_messages([("human", "{ticket}"), ("ai", "{label}")])
few_shot = FewShotChatMessagePromptTemplate(examples=examples, example_prompt=example_prompt)
final = ChatPromptTemplate.from_messages([("system", "Classify the ticket into one label."), few_shot, ("human",
        "{ticket}")])
(final | llm).invoke({"ticket": "Unable to log in to net banking"})
```

Example selectors

With many examples, choose the most relevant ones per input instead of sending all of them:

Selector	Picks examples by
SemanticSimilarityExampleSelector	embedding similarity to the input (vector store) — most useful
MaxMarginalRelevanceExampleSelector	similar but diverse
LengthBasedExampleSelector	fit within a length budget
NGramOverlapExampleSelector	n-gram overlap

example_selector.py

```
from langchain_core.example_selectors import SemanticSimilarityExampleSelector
from langchain_chroma import Chroma
selector = SemanticSimilarityExampleSelector.from_examples(examples, emb, Chroma, k=2)
few_shot = FewShotChatMessagePromptTemplate(example_selector=selector, example_prompt=example_prompt,
        input_variables=["ticket"])
```

3.3 Prompt management

- Keep prompts in version control (prompts.py or YAML) or in the **LangSmith prompt hub** (client.pull_prompt('name'), push with commits) for collaboration and versioning.
- Prompt engineering basics still apply: role, task, context, constraints, output format, examples, an 'I don't know' path, and delimiters around data.
- Test prompts against a dataset (LangSmith evaluation) before changing them in production.

CHAPTER 4

Output Parsers & Structured Output

StrOutputParser • JSON • Pydantic • list/enum/datetime parsers • with_structured_output • include_raw • fixing & retry parsers • streaming parsing

4.1 Why parse outputs?

LLMs return text, but programs need typed data (a label, a JSON object, a list). Two approaches: **(1) structured output / tool calling** — the provider natively constrains output to a schema (most reliable, preferred); **(2) output parsers** — instruct the model with format instructions and parse the text.

4.2 with_structured_output (preferred)

structured_output.py

```
from pydantic import BaseModel, Field
from typing import Literal, Optional
class Ticket(BaseModel):
    """Classification of a customer support ticket."""          # docstring becomes the schema description
    category: Literal["billing", "technical", "account", "other"] = Field(description="ticket category")
    priority: Literal["P1", "P2", "P3"]
    summary: str = Field(description="one-line summary")
    order_id: Optional[str] = None

extractor = llm.with_structured_output(Ticket)                  # returns a Ticket instance
t = extractor.invoke("My payment for order ORD-77 was debited twice, please refund urgently!")
print(t.category, t.priority, t.order_id)

raw = llm.with_structured_output(Ticket, include_raw=True).invoke("...")
# {'raw': AIMessage, 'parsed': Ticket | None, 'parsing_error': Exception | None}  → handle failures yourself
```

Schema type	Returns
Pydantic BaseModel	validated model instance (recommended)
TypedDict	dict
JSON schema dict	dict

Methods (method= argument): **function_calling** / tool calling (default on many providers), **json_schema** (provider-native strict structured output) and **json_mode**. Tips: describe fields well, use Literal/enums for labels, keep schemas flat where possible, and validate.

4.3 Output parsers

Parser	Output	Notes
StrOutputParser	str	extracts .content from AIMessage — the most used parser in chains; streams token by token
JsonOutputParser(pydantic_object=...)	dict	streams partial JSON; get_format_instructions()
PydanticOutputParser(pydantic_object=M)	M instance	format instructions + validation
CommaSeparatedListOutputParser	list[str]	simple lists
EnumOutputParser / DatetimeOutputParser	Enum / datetime	
XMLOutputParser, YamlOutputParser	dict / model	
JsonOutputToolsParser / PydanticToolsParser	tool calls → dict / models	parse tool_calls
OutputFixingParser (classic)	—	on failure, asks an LLM to fix the malformed output
RetryOutputParser (classic)	—	re-prompts with the original prompt + error

parsers.py

```
from langchain_core.output_parsers import StrOutputParser, JsonOutputParser, PydanticOutputParser
parser = PydanticOutputParser(pydantic_object=Ticket)
prompt = ChatPromptTemplate.from_messages([
    ("system", "Classify the ticket.\n{format_instructions}"), ("human", "{ticket}")
]).partial(format_instructions=parser.get_format_instructions())
chain = prompt | llm | parser
chain.invoke({"ticket": "App crashes on login"})

for partial in (prompt | llm | JsonOutputParser()).stream({"ticket": "..."}): print(partial)   # growing dict
```

Writing a custom parser

custom_parser.py

```
from langchain_core.runnables import RunnableLambda
def to_label(msg) -> str:
    text = msg.content.strip().lower()
    return next((l for l in ["billing", "technical", "account"] if l in text), "other")
chain = prompt | llm | RunnableLambda(to_label)       # any function can be a parser
```

CHAPTER 5

Runnables & LCEL (LangChain Expression Language)

Runnable protocol • invoke / batch / stream / async / astream_events • RunnableSequence • RunnableParallel • RunnablePassthrough & assign • RunnableLambda • RunnableBranch & routing • @chain • RunnableConfig • bind • retry & fallbacks • configurable fields & alternatives • schemas • graph visualisation • coercion rules

5.1 The Runnable protocol — the heart of LangChain

Almost everything in LangChain implements **Runnable**: prompts, chat models, output parsers, retrievers, tools, chains, and even compiled LangGraph graphs. Because they share one interface, they can be composed and they all get batching, streaming, async, retries and tracing for free.



Figure: Every Runnable exposes the same methods.

Method	Input → Output	Notes
invoke(input, config=None)	one → one	synchronous call
batch(inputs, config, return_exceptions=False)	list → list	runs in parallel threads; max_concurrency in config
batch_as_completed	list → iterator of (index, output)	results as they finish
stream(input)	one → iterator of chunks	streams if every step supports it (parsers that need the full input block streaming)
ainvoke / abatch / astream	async versions	use in FastAPI / asyncio
astream_events(input, version='v2')	async iterator of events	on_chat_model_stream, on_tool_start, on_retriever_end… from every inner step
transform / atransform	iterator → iterator	streaming input

Input / output types of common runnables

Component	Input	Output
Prompt template	dict	PromptValue
Chat model	str | list[messages] | PromptValue	AIMessage
Output parser	AIMessage (or str)	depends on parser
Retriever	str (query)	list[Document]
Tool	str | dict | ToolCall	tool output / ToolMessage
Compiled LangGraph	state dict	state dict

5.2 LCEL composition primitives

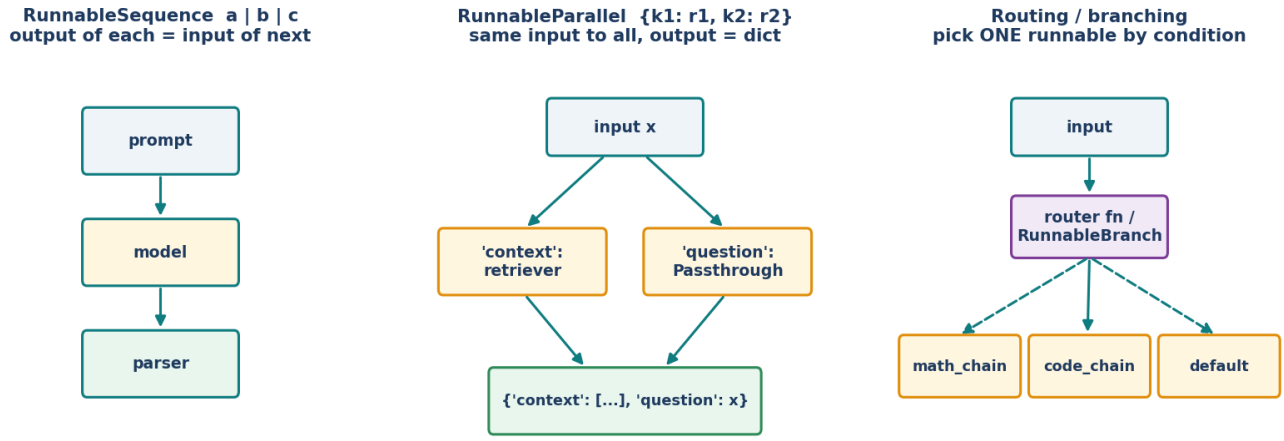


Figure: Sequence, parallel and routing — the three ways to compose runnables.

RunnableSequence: the | operator

sequence.py

```
chain = prompt | llm | StrOutputParser()          # RunnableSequence(first=prompt, middle=[llm], last=parser)
chain.invoke({"topic": "vector databases"})
# equivalent: prompt.pipe(llm).pipe(StrOutputParser())
```

RunnableParallel: run branches on the same input

parallel.py

```
from langchain_core.runnables import RunnableParallel
summary = ChatPromptTemplate.from_template("Summarise: {text}") | llm | StrOutputParser()
keywords = ChatPromptTemplate.from_template("5 keywords for: {text}") | llm | StrOutputParser()
both = RunnableParallel(summary=summary, keywords=keywords)    # or simply {"summary": summary, "keywords": keywords}
both.invoke({"text": long_text})                               # {'summary': '...', 'keywords': '...'} — runs concurrently
```

RunnablePassthrough and .assign

passthrough_assign.py

```
from langchain_core.runnables import RunnablePassthrough
# pass the input through unchanged (often the question in RAG)
rag = {"context": retriever | format_docs, "question": RunnablePassthrough()} | prompt | llm | StrOutputParser()

# .assign: keep all existing keys and ADD computed ones
chain = RunnablePassthrough.assign(context=lambda x: format_docs(retriever.invoke(x["question"]))) | prompt | llm
chain.invoke({"question": "Leave policy?"})                    # input dict gains 'context'

# itemgetter is handy to pick keys
from operator import itemgetter
chain = {"context": itemgetter("question") | retriever, "question": itemgetter("question"), "lang": itemgetter("lang")} |
        prompt | llm
```

Primitive	Meaning
RunnablePassthrough()	output = input
RunnablePassthrough.assign(k=r)	output = {**input, k: r(input)}
RunnablePick / .pick('k')	select keys from a dict output
itemgetter('k')	plain Python function → coerced to RunnableLambda

RunnableLambda and @chain: plain Python inside chains

lambda_chain.py

```
from langchain_core.runnables import RunnableLambda, chain
def format_docs(docs): return "\n\n".join(d.page_content for d in docs)
fmt = RunnableLambda(format_docs)                # now has invoke/batch/stream...
upper = RunnableLambda(lambda s: s.upper())

@chain                                           # decorator turns a function into a Runnable (and traces it as one step)
def answer_with_check(question: str) -> str:
    draft = (prompt | llm | StrOutputParser()).invoke({"question": question})
    return draft if len(draft) < 2000 else draft[:2000]
answer_with_check.invoke("What is RAG?")
# A lambda with ONE argument receives the input; to also get config: def f(x, config): ...
```

Routing and RunnableBranch

routing.py

```
from langchain_core.runnables import RunnableBranch
class Route(BaseModel): destination: Literal["math", "code", "general"]
router = ChatPromptTemplate.from_template("Classify: {q}") | llm.with_structured_output(Route)

# (a) recommended: custom routing function returning a runnable
def route(info):
    return {"math": math_chain, "code": code_chain}.get(info["route"].destination, general_chain)
full = {"route": router, "q": itemgetter("q")} | RunnableLambda(route)

# (b) RunnableBranch: list of (condition, runnable) pairs + default
branch = RunnableBranch((lambda x: "sql" in x["q"].lower(), sql_chain), (lambda x: "python" in x["q"].lower(), code_chain),
        general_chain)
```

When a RunnableLambda *returns a Runnable*, LangChain invokes it with the same input — that is how dynamic routing works. For complex routing with loops, prefer LangGraph conditional edges.

Coercion rules (how LCEL treats Python objects)

You write	LCEL turns it into
a dict {k: runnable_or_fn}	RunnableParallel
a function / lambda	RunnableLambda
a string?	not allowed — wrap it in a prompt

You write	LCEL turns it into
runnable | dict	sequence ending in a parallel step
dict | runnable	needs at least one side to be a Runnable for | to work

5.3 RunnableConfig — controlling execution

config.py

```
config = {
    "run_name": "ticket-classifier",                # name in LangSmith traces
    "tags": ["prod", "v2"], "metadata": {"user_id": "u42"},
    "callbacks": [MyHandler()],                     # callback handlers
    "max_concurrency": 4,                           # for batch
    "recursion_limit": 25,                          # for LangGraph
    "configurable": {"thread_id": "t1", "model": "gemini-2.5-flash"},
}
chain.invoke(inputs, config=config)
named = chain.with_config(run_name="rag", tags=["rag"])      # bake config into the runnable
```

Config is automatically propagated to all child runnables (in Python 3.11+ even through async code), so tags/callbacks set at the top appear on every inner step. Inside a RunnableLambda, accept a *config* parameter to read it.

5.4 Wrapping runnables: bind, retry, fallbacks, configurable

wrappers.py

```
llm_stop = llm.bind(stop=["\nObservation"])             # fix call-time kwargs
llm_tools = llm.bind_tools([get_weather], tool_choice="auto")   # bind tools (special case of bind)

safe = chain.with_retry(retry_if_exception_type=(TimeoutError,), stop_after_attempt=3,
        wait_exponential_jitter=True)
resilient = primary_chain.with_fallbacks([backup_chain], exceptions_to_handle=(Exception,))

from langchain_core.runnables import ConfigurableField
tunable = llm.configurable_fields(temperature=ConfigurableField(id="temp", name="Temperature"))
tunable.with_config(configurable={"temp": 0.9}).invoke("Write a slogan")

switchable = gemini.configurable_alternatives(ConfigurableField(id="llm"), default_key="gemini", openai=gpt)
(prompt | switchable).invoke({...}, config={"configurable": {"llm": "openai"}})

typed = chain.with_types(input_type=MyInput)                  # declare schema
```

Method	Purpose
bind(**kwargs)	attach fixed invocation arguments
with_retry(...)	retry on exceptions with backoff
with_fallbacks([...])	try alternatives if the primary fails (model outage, rate limit)
with_config(...)	default config (name, tags, callbacks)
configurable_fields / configurable_alternatives	change parameters or swap components at runtime
with_listeners(on_start, on_end, on_error)	lifecycle hooks
with_types / input_schema / output_schema	Pydantic schemas of inputs/outputs (used by servers/UIs)
as_tool(name, description)	turn any runnable into a tool for agents
assign / pick / map	dict manipulation; map applies to each list element

5.5 Streaming deep dive

streaming.py

```
for tok in (prompt | llm | StrOutputParser()).stream({"topic": "LCEL"}): print(tok, end="")

async for ev in rag_chain.astream_events({"question": "..."}, version="v2"):
    if ev["event"] == "on_chat_model_stream": print(ev["data"]["chunk"].content, end="")
    elif ev["event"] == "on_retriever_end": print("\n[retrieved]", len(ev["data"]["output"]), "docs")
```

A chain streams end-to-end only if every step can process input incrementally (prompt → model → StrOutputParser does). A RunnableLambda over the full output breaks streaming — write it as a generator function to keep streaming.

5.6 Inspecting and visualising chains

inspect.py

```
chain.get_graph().print_ascii()          # ASCII diagram of the chain
print(chain.get_graph().draw_mermaid())   # Mermaid text; draw_mermaid_png() for an image
chain.input_schema.model_json_schema(); chain.output_schema.model_json_schema()
from langchain_core.globals import set_debug, set_verbose
set_debug(True)                           # print every input/output of every step
```

Interview tip: Why LCEL? One syntax gives: automatic parallelism in RunnableParallel and batch, streaming of intermediate tokens, async support, retries/fallbacks, config propagation and full LangSmith tracing. Its limit: no cycles or shared mutable state — that is what LangGraph is for.

CHAPTER 6

Chains: Legacy and Modern

What a chain is • LLMChain → prompt | llm • SequentialChain • RetrievalQA & ConversationalRetrievalChain • document chains (stuff, map-reduce, refine, map-rerank) • create_stuff_documents_chain • create_retrieval_chain • create_history_aware_retriever • summarisation • extraction & tagging

6.1 Chains, then and now

A **chain** is a fixed sequence of calls (LLM, retriever, tools, functions). Early LangChain shipped many chain classes; these now live in **langchain-classic** and are replaced by LCEL compositions or LangGraph. You must recognise both.

Legacy class (langchain-classic)	What it did	Modern equivalent
LLMChain(llm, prompt)	prompt → LLM → text	prompt | llm | StrOutputParser()
SimpleSequentialChain / SequentialChain	output of chain A feeds chain B	LCEL sequence with RunnablePassthrough.assign, or LangGraph nodes
TransformChain	arbitrary Python transform	RunnableLambda
RouterChain / MultiPromptChain	route to sub-chains	routing function / RunnableBranch / LangGraph conditional edges
ConversationChain + Memory	chat with memory	RunnableWithMessageHistory or LangGraph checkpointer
RetrievalQA	retrieve then answer	create_retrieval_chain or custom LCEL RAG
ConversationalRetrievalChain	condense question + RAG	create_history_aware_retriever + create_retrieval_chain, or LangGraph
load_summarize_chain / MapReduceDocumentsChain / RefineDocumentsChain	summarise many docs	LCEL map-reduce or LangGraph with Send
LLMMathChain, APIChain, SQLDatabaseChain	math/API/SQL	tools + agent (safer)
AgentExecutor	agent loop	create_agent / LangGraph

llmchain_vs_lcel.py

```
# LEGACY (still appears in tests/old code)
from langchain_classic.chains import LLMChain            # older code: from langchain.chains import LLMChain
chain = LLMChain(llm=llm, prompt=prompt); chain.run(topic="AI")      # .run / .predict / __call__ are
        deprecated

# MODERN
chain = prompt | llm | StrOutputParser(); chain.invoke({"topic": "AI"})
```

Sequential chains in LCEL

sequential.py

```
outline = ChatPromptTemplate.from_template("Outline a blog on {topic}") | llm | StrOutputParser()
write = ChatPromptTemplate.from_template("Write the blog from this outline:\n{outline}") | llm |
        StrOutputParser()
review = ChatPromptTemplate.from_template("Improve clarity:\n{draft}") | llm | StrOutputParser()
pipeline = (RunnablePassthrough.assign(outline=outline)          # {'topic', 'outline'}
            | RunnablePassthrough.assign(draft=write)            # + 'draft'
            | RunnablePassthrough.assign(final=review))          # + 'final' — keeps all intermediates
out = pipeline.invoke({"topic": "LangGraph"}); print(out["final"])
```

6.2 Document chains: combining many documents

Strategy	How	Pros	Cons
Stuff	put all docs into one prompt	1 call, best context	limited by context window
Map-reduce	summarise/answer per doc (map, parallel) → combine (reduce, possibly recursively)	scales, parallel	loses cross-doc context, more calls
Refine	answer with doc 1, then refine with doc 2, 3… sequentially	keeps a running context	slow, sequential, order-sensitive
Map-rerank	answer per doc with a score; take best	good for single-fact QA	no combination

retrieval_chain_helpers.py

```
from langchain_classic.chains.combine_documents import create_stuff_documents_chain
from langchain_classic.chains import create_retrieval_chain, create_history_aware_retriever
qa_prompt = ChatPromptTemplate.from_messages([("system", "Answer only from context:\n\n{context}"),
        MessagesPlaceholder("chat_history"), ("human", "{input}")])
doc_chain = create_stuff_documents_chain(llm, qa_prompt)            # expects 'context' = list[Document]
contextualize = ChatPromptTemplate.from_messages([("system", "Rewrite the question to be standalone."),
        MessagesPlaceholder("chat_history"), ("human", "{input}")])
hist_retriever = create_history_aware_retriever(llm, retriever, contextualize)
rag = create_retrieval_chain(hist_retriever, doc_chain)
r = rag.invoke({"input": "And for interns?", "chat_history": history})
r["answer"], r["context"]                                           # answer + retrieved docs (citations)
```

These helper constructors were in *langchain.chains* before v1 and are in *langchain_classic.chains* now. Note the key names: input key **input**, output keys **answer** and **context**.

Map-reduce summarisation in LCEL

map_reduce.py

```
map_chain = ChatPromptTemplate.from_template("Summarise this part:\n{doc}") | llm | StrOutputParser()
reduce_chain = ChatPromptTemplate.from_template("Combine these summaries into one:\n{summaries}") | llm |
        StrOutputParser()
def map_reduce(docs):
    parts = map_chain.batch([{"doc": d.page_content} for d in docs], config={"max_concurrency": 5})   # MAP (parallel)
    return reduce_chain.invoke({"summaries": "\n\n".join(parts)})                                     # REDUCE
summary = RunnableLambda(map_reduce).invoke(chunks)
```

6.3 Extraction and tagging chains

extraction_tagging.py

```
class Person(BaseModel):
    name: str; role: Optional[str] = None; company: Optional[str] = None
class People(BaseModel): people: list[Person]
extract = ChatPromptTemplate.from_messages([("system", "Extract all people. If unknown leave null; never invent."),
        ("human", "{text}")]) | llm.with_structured_output(People)
extract.invoke({"text": "Nikita Dash leads the GenAI team at LTIMindtree; Gopal reviews."})

class Tag(BaseModel): sentiment: Literal["pos", "neg", "neu"]; language: str; urgent: bool
tagger = ChatPromptTemplate.from_template("Tag the message:\n{msg}") | llm.with_structured_output(Tag)
tagger.batch([{"msg": m} for m in messages])
```

CHAPTER 7

Memory & Chat History

Why LLMs are stateless • legacy memory classes • chat message histories • RunnableWithMessageHistory • trimming & summarising • modern approach with LangGraph

7.1 LLMs are stateless

Every API call is independent; the model remembers nothing. 'Memory' means **your app stores past messages and sends the relevant ones again** each turn. Design questions: what to store, where (RAM, Redis, SQL), how much to send (window, summary, retrieval), and how to scope it (per user/session).

7.2 Legacy memory classes (langchain-classic) — recognise them

Class	Keeps	Trade-off
ConversationBufferMemory	entire history	simple; grows without bound
ConversationBufferWindowMemory(k)	last k exchanges	bounded; forgets older facts
ConversationTokenBufferMemory	last N tokens	token-aware window
ConversationSummaryMemory	running LLM summary	compact; costs LLM calls; loses detail
ConversationSummaryBufferMemory	recent messages verbatim + summary of older	best balance
ConversationEntityMemory	facts about entities	structured but brittle
VectorStoreRetrieverMemory	embeds past turns; retrieves relevant ones	long-term, semantic recall

legacy_memory.py

```
# LEGACY pattern
from langchain_classic.memory import ConversationBufferWindowMemory
from langchain_classic.chains import ConversationChain
conv = ConversationChain(llm=llm, memory=ConversationBufferWindowMemory(k=3)); conv.predict(input="Hi, I'm
        Mansavi")
```

7.3 Chat message histories and RunnableWithMessageHistory

A **BaseChatMessageHistory** stores messages for one session: InMemoryChatMessageHistory, FileChatMessageHistory, RedisChatMessageHistory, SQLChatMessageHistory, etc. **RunnableWithMessageHistory** wraps any chain: before invoke it loads history for the session_id, after invoke it appends the new human + AI messages.

message_history.py

```
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory
store = {}
def get_history(session_id: str):
    if session_id not in store: store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]
prompt = ChatPromptTemplate.from_messages([("system", "You are helpful."), MessagesPlaceholder("history"), ("human",
        "{input}")])
chat = RunnableWithMessageHistory(prompt | llm, get_history, input_messages_key="input", history_messages_key="history")
cfg = {"configurable": {"session_id": "mansavi-1"}}
chat.invoke({"input": "My name is Mansavi"}, config=cfg)
chat.invoke({"input": "What is my name?"}, config=cfg)          # → "Mansavi"
```

Keeping history within the context window

trimming.py

```
from langchain_core.messages import trim_messages
trimmer = trim_messages(max_tokens=1000, strategy="last", token_counter=llm, include_system=True, start_on="human",
        allow_partial=False)
chain = RunnablePassthrough.assign(history=itemgetter("history") | trimmer) | prompt | llm
```

Interview tip: Modern recommendation: for new chatbots/agents use LangGraph persistence — a **checkpointer** keyed by thread_id for short-term memory and a **Store** for long-term memory (Part II, Chapter 15). create_agent accepts checkpointer= and store= directly, and SummarizationMiddleware handles long histories.

CHAPTER 8

Documents, Loaders, Splitters, Vector Stores & Retrievers (RAG in LangChain)

Document object • loaders • text splitters • embeddings • vector stores • retrievers & search types • advanced retrievers • indexing API • full RAG chain • conversational RAG

8.1 The Document object

document.py

```
from langchain_core.documents import Document
doc = Document(page_content="Employees get 12 sick leaves per year.", metadata={"source": "leave.pdf", "page": 7, "dept":
        "HR"}, id="leave-7")
```

8.2 Document loaders

Source	Loader (mostly langchain_community.document_loaders)
PDF	PyPDFLoader (page per Document), PyMuPDFLoader, PDFPlumberLoader, UnstructuredPDFLoader
Text / Markdown	TextLoader, UnstructuredMarkdownLoader
CSV / Excel	CSVLoader (row per Document), UnstructuredExcelLoader
Word / PowerPoint	Docx2txtLoader, UnstructuredWordDocumentLoader, UnstructuredPowerPointLoader
Web	WebBaseLoader (BeautifulSoup), RecursiveUrlLoader, SitemapLoader, AsyncHtmlLoader
Folders	DirectoryLoader(path, glob='**/*.pdf', loader_cls=PyPDFLoader)
JSON	JSONLoader(jq_schema=...)
Others	Confluence, Notion, SharePoint, Google Drive, GitHub, YouTube transcripts, S3, databases (SQLDatabaseLoader)

loaders.py

```
from langchain_community.document_loaders import PyPDFLoader, DirectoryLoader, WebBaseLoader
pages = PyPDFLoader("policy.pdf").load()                    # list[Document], metadata has source & page
docs = DirectoryLoader("kb/", glob="**/*.pdf", loader_cls=PyPDFLoader, show_progress=True).load()
for d in PyPDFLoader("huge.pdf").lazy_load(): ...           # stream large files
web = WebBaseLoader("https://example.com/faq").load()
```

8.3 Text splitters

Splitter (langchain_text_splitters)	Splits by	Use
RecursiveCharacterTextSplitter	tries separators ['\n\n', '\n', ' ', ''] recursively until chunks fit	default for text
CharacterTextSplitter	one separator	simple
TokenTextSplitter / from_tiktoken_encoder	tokens	precise token budgets
MarkdownHeaderTextSplitter	#, ## headers → header metadata	docs/wikis
HTMLHeaderTextSplitter / HTMLSectionSplitter	HTML structure	web pages
RecursiveCharacterTextSplitter.from_language(Language.PYTHON)	code syntax (classes/functions)	code RAG
RecursiveJsonSplitter	JSON structure	APIs
SemanticChunker (langchain_experimental)	embedding similarity breakpoints	topic-coherent chunks

splitters.py

```
from langchain_text_splitters import RecursiveCharacterTextSplitter, MarkdownHeaderTextSplitter
splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=100, add_start_index=True)
chunks = splitter.split_documents(pages)              # keeps & extends metadata
texts = splitter.split_text(raw_string)              # list[str]
md = MarkdownHeaderTextSplitter(headers_to_split_on=[("#", "h1"), ("##", "h2")]).split_text(markdown)
```

chunk_size is measured by **length_function** (characters by default). chunk_overlap repeats text between neighbouring chunks so sentences on boundaries are not lost (10–20% typical). Tune using retrieval evaluation.

8.4 Vector stores

Store	Package	Notes
Chroma	langchain-chroma	local, persistent, metadata filters — great for dev
FAISS	langchain-community (faiss-cpu)	in-memory library; save_local/load_local
InMemoryVectorStore	langchain-core	tests and demos
Qdrant / Weaviate / Milvus / Pinecone	partner packages	production servers / managed
PGVector	langchain-postgres	Postgres + vectors
Elasticsearch / OpenSearch / Azure AI Search	partner packages	hybrid keyword + vector

vector_stores.py

```
from langchain_chroma import Chroma
vs = Chroma.from_documents(chunks, emb, collection_name="kb", persist_directory="./chroma_db")
# or: vs = Chroma(collection_name="kb", embedding_function=emb, persist_directory="./chroma_db"); vs.add_documents(chunks,
        ids=ids)
vs.similarity_search("sick leave", k=4)                                  # list[Document]
vs.similarity_search_with_score("sick leave", k=4)                       # (Document, distance) — lower = closer for Chroma
vs.similarity_search("leave", k=4, filter={"dept": "HR"})                # metadata filter
vs.max_marginal_relevance_search("leave", k=4, fetch_k=20, lambda_mult=0.5)
vs.delete(ids=["leave-7"])

from langchain_community.vectorstores import FAISS
f = FAISS.from_documents(chunks, emb); f.save_local("faiss_idx")
f = FAISS.load_local("faiss_idx", emb, allow_dangerous_deserialization=True)   # only for files you trust
```

8.5 Retrievers

A **retriever** is a Runnable: str → list[Document]. It need not be a vector store (BM25, web search, SQL, Wikipedia…). **vectorstore.as_retriever()** converts a store into one.

search_type	search_kwargs	Behaviour
'similarity' (default)	{'k': 4, 'filter': {…}}	top-k nearest
'mmr'	{'k': 4, 'fetch_k': 20, 'lambda_mult': 0.5}	relevant + diverse
'similarity_score_threshold'	{'score_threshold': 0.5, 'k': 8}	only results above a relevance score (may return none → 'I don't know')

Advanced retrievers

Retriever	Idea	Import (v1)
BM25Retriever	keyword search (rank_bm25)	langchain_community.retrievers
EnsembleRetriever	combine retrievers with weighted Reciprocal Rank Fusion → hybrid search	langchain_classic.retrievers
MultiQueryRetriever	LLM generates query variants; union of results	langchain_classic.retrievers
ContextualCompressionRetriever	post-process docs with a compressor: LLMChainExtractor, EmbeddingsFilter, cross-encoder reranker (CrossEncoderReranker), Cohere/Flashrank rerank	langchain_classic.retrievers
ParentDocumentRetriever	search small child chunks, return parent chunks (needs docstore)	langchain_classic.retrievers
MultiVectorRetriever	multiple vectors per doc (summaries, hypothetical questions, table summaries)	langchain_classic.retrievers
SelfQueryRetriever	LLM turns natural language into query + metadata filter	langchain_classic.retrievers
TimeWeightedVectorStoreRetriever	recency-weighted relevance	langchain_classic.retrievers
Custom	subclass BaseRetriever and implement _get_relevant_documents	langchain_core.retrievers

advanced_retrievers.py

```
from langchain_community.retrievers import BM25Retriever
from langchain_classic.retrievers import EnsembleRetriever, ContextualCompressionRetriever, MultiQueryRetriever
from langchain_classic.retrievers.document_compressors import CrossEncoderReranker
from langchain_community.cross_encoders import HuggingFaceCrossEncoder

bm25 = BM25Retriever.from_documents(chunks); bm25.k = 10
dense = vs.as_retriever(search_kwargs={"k": 10})
hybrid = EnsembleRetriever(retrievers=[bm25, dense], weights=[0.4, 0.6])          # weighted RRF
rerank = CrossEncoderReranker(model=HuggingFaceCrossEncoder(model_name="BAAI/bge-reranker-base"), top_n=4)
retriever = ContextualCompressionRetriever(base_compressor=rerank, base_retriever=hybrid)
mq = MultiQueryRetriever.from_llm(retriever=dense, llm=llm)                        # query expansion
```

custom_retriever.py

```
from langchain_core.retrievers import BaseRetriever
from langchain_core.callbacks import CallbackManagerForRetrieverRun
class KeywordRetriever(BaseRetriever):
    docs: list[Document]; k: int = 3
    def _get_relevant_documents(self, query: str, *, run_manager: CallbackManagerForRetrieverRun) ->
        list[Document]:
        return [d for d in self.docs if query.lower() in d.page_content.lower()][: self.k]
```

8.6 Indexing API (keeping the vector store in sync)

Re-running ingestion naively creates duplicates. The **indexing API** (index() with a **RecordManager**, in langchain-classic in v1) hashes each document, skips unchanged ones, and with cleanup='incremental' or 'full' deletes vectors for removed/changed source documents. Alternative: use stable IDs (hash of source + chunk index) and upsert.

8.7 Putting it together: a complete RAG chain

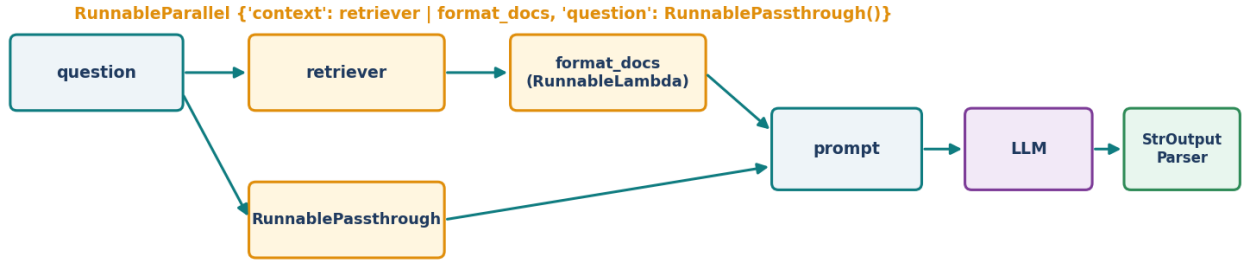


Figure: Data flow of the classic LCEL RAG chain.

rag_chain.py

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough, RunnableParallel

def format_docs(docs): return "\n\n".join(f"[{d.metadata.get('source')} p{d.metadata.get('page')}] {d.page_content}" for d
        in docs)
prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer ONLY from the context. If the answer is not there, say you don't know. Cite [source
        pN].\n\nContext:\n{context}"),
    ("human", "{question}")])

rag = ({"context": retriever | format_docs, "question": RunnablePassthrough()} | prompt | llm | StrOutputParser())
print(rag.invoke("How many sick leaves do I get?"))

# return sources too
rag_with_sources = RunnableParallel(docs=retriever, question=RunnablePassthrough()).assign(
    answer=(lambda x: {"context": format_docs(x["docs"]), "question": x["question"]}) | prompt | llm | StrOutputParser())
out = rag_with_sources.invoke("How many sick leaves do I get?"); out["answer"], [d.metadata for d in out["docs"]]
```

Interview tip: The same RAG can be built as an **agent**: wrap the retriever as a tool (retriever.as_tool(...) or @tool) and give it to create_agent — the model decides when (and how often) to search. Use the chain for predictable single-shot QA, the agent for multi-step/ambiguous questions, and LangGraph for CRAG/Self-RAG control flow.

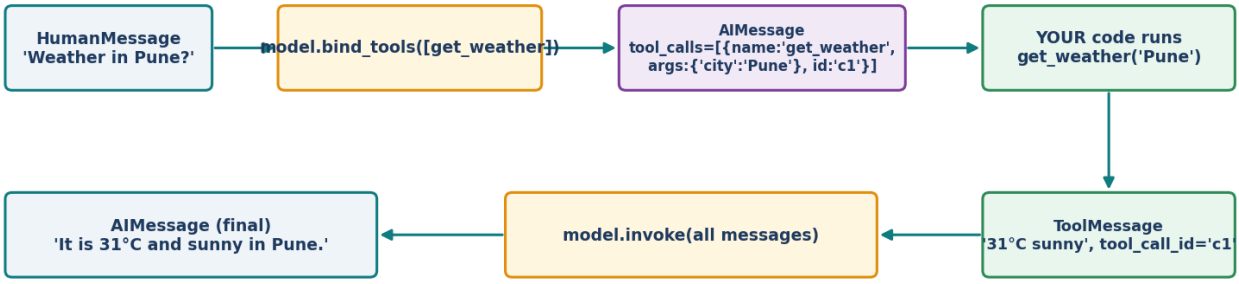
CHAPTER 9

Tools & Tool Calling

What a tool is • @tool • args_schema • StructuredTool • BaseTool subclass • runnables as tools • tool artifacts • injected args & ToolRuntime • bind_tools & tool_choice • parallel tool calls • executing tool calls • built-in toolkits • MCP tools • error handling

9.1 What is a tool?

A **tool** = a function + a **name** + a **description** + an **input schema**. The schema and description are sent to the model; the model decides whether to call it and with which arguments. Tools are Runnables (invoke with a dict of args or with a ToolCall).



The model never executes tools — it only REQUESTS them. Your code (or ToolNode / create_agent) executes and returns ToolMessages.

Figure: The complete tool-calling round trip.

9.2 Creating tools

create_tools.py

```
from langchain.tools import tool                    # also: from langchain_core.tools import tool
@tool
def get_weather(city: str, unit: str = "celsius") -> str:
    """Get the current weather for a city. Use for any weather question."""
    return f"31 {unit} and sunny in {city}"
get_weather.name, get_weather.description, get_weather.args       # 'get_weather', docstring, {'city':..., 'unit':...}
get_weather.invoke({"city": "Pune"})

class SearchArgs(BaseModel):
    query: str = Field(description="search keywords"); max_results: int = Field(5, ge=1, le=20)
@tool("kb_search", args_schema=SearchArgs, return_direct=False)
def kb_search(query: str, max_results: int = 5) -> str:
    """Search the internal knowledge base for policies."""
    return format_docs(retriever.invoke(query)[:max_results])

@tool(parse_docstring=True)          # Google-style docstring Args: become parameter descriptions
def convert(amount: float, to: str) -> float:
    """Convert INR to another currency.
    Args:
        amount: amount in INR
        to: target currency code like USD
    """
    return amount * RATES[to]
```

Way to create	When
@tool decorator	most cases

Way to create	When
StructuredTool.from_function(func=..., coroutine=..., name=..., args_schema=...)	build tools programmatically / sync + async versions
Subclass BaseTool (name, description, args_schema, _run, _arun)	full control, stateful clients
runnable.as_tool(name=, description=, args_schema=)	turn a chain into a tool
retriever as tool (create_retriever_tool in classic, or @tool wrapper)	agentic RAG
Toolkits (SQLDatabaseToolkit, FileManagementToolkit, GmailToolkit...)	pre-built groups
MCP: MultiServerMCPClient(...).get_tools()	tools exposed by MCP servers (langchain-mcp-adapters)

Advanced tool features

advanced_tools.py

```
# 1) content + artifact: send short text to the model, keep raw data for your app
@tool(response_format="content_and_artifact")
def fetch_rows(sql: str):
    """Run a read-only SQL query."""
    rows = db.run(sql); return f"{len(rows)} rows returned", rows      # (content, artifact)

# 2) arguments the MODEL must not see/fill (user id, DB session) — injected at runtime
from langchain_core.tools import InjectedToolArg
from typing import Annotated
@tool
def my_orders(status: str, user_id: Annotated[str, InjectedToolArg]) -> list:
    """List the current user's orders by status."""
    return orders_for(user_id, status)

# 3) In v1 agents, tools can receive a ToolRuntime (state, context, store, stream writer, config)
from langchain.tools import ToolRuntime
@tool
def remember(fact: str, runtime: ToolRuntime) -> str:
    """Save a fact about the user."""
    runtime.store.put(("users", runtime.context.user_id), "fact", {"text": fact}); return "saved"
```

Also: in LangGraph, InjectedState and InjectedStore annotations inject graph state/store into tools, and a tool can return a **Command** to update graph state directly (e.g. a handoff tool).

9.3 Binding tools to a model and executing calls

manual_tool_loop.py

```
llm_t = llm.bind_tools([get_weather, kb_search], tool_choice="auto")   # "any"/"required" = must call some tool; or a tool
        name; parallel_tool_calls=False on some providers
ai = llm_t.invoke("Weather in Pune and Delhi?")
ai.tool_calls   # [{'name':'get_weather','args':{'city':'Pune'},'id':'c1','type':'tool_call'}, {...'Delhi'...}]  ← parallel
        calls

msgs = [HumanMessage("Weather in Pune and Delhi?"), ai]
tools_by_name = {t.name: t for t in [get_weather, kb_search]}
for call in ai.tool_calls:
    msgs.append(tools_by_name[call["name"]].invoke(call))          # invoking with a ToolCall returns a ToolMessage
final = llm_t.invoke(msgs); print(final.content)
```

Interview tip: Good tool design: clear name, a description saying *when to use and when not to*, few typed arguments with descriptions, small structured outputs, idempotent and safe; return error text instead of raising so the model can recover; gate write/irreversible tools behind human approval.

CHAPTER 10

Agents in LangChain

What an agent is • legacy agents (AgentExecutor, ReAct prompt, agent types) • create_agent (v1) • models & dynamic models • tools • system prompts & dynamic prompts • structured responses • state & context • memory • streaming • middleware (hooks, prebuilt, custom) • guardrails

10.1 What is an agent?

An agent uses an LLM to decide which actions to take: call a tool, observe the result, decide again — looping until it can answer. The core algorithm (since 2022): give the model tools; call it; if it requests tools, execute them and append results; repeat; stop when it answers without tool calls.[5]

10.2 Legacy agents (recognise in old code)

Legacy API	Notes
initialize_agent(tools, llm, agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION)	oldest API; text-based ReAct parsing ('Thought: … Action: … Action Input: … Observation:')
create_react_agent(llm, tools, prompt) + AgentExecutor	LangChain ReAct agent with a hub prompt (hwchase17/react); prompt needs {tools}, {tool_names}, {agent_scratchpad}
create_tool_calling_agent / create_openai_tools_agent + AgentExecutor	uses native tool calling; prompt needs MessagesPlaceholder('agent_scratchpad')
AgentExecutor(agent, tools, verbose=True, max_iterations=..., handle_parsing_errors=True, return_intermediate_steps=True)	the runtime loop; now in langchain-classic
Agent types: zero-shot ReAct, structured chat, conversational, self-ask-with-search, OpenAI functions	historical; replaced by tool calling

legacy_agent.py

```
# LEGACY tool-calling agent (langchain-classic)
from langchain_classic.agents import AgentExecutor, create_tool_calling_agent
prompt = ChatPromptTemplate.from_messages([("system", "You are helpful."), ("human", "{input}"),
        MessagesPlaceholder("agent_scratchpad")])
agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True, max_iterations=5)
executor.invoke({"input": "Weather in Pune?"})          # {'input':..., 'output': ...}
```

10.3 create_agent — the v1 standard

create_agent builds a LangGraph graph implementing the agent loop; it replaces langgraph.prebuilt.create_react_agent.[2] Input/output is a state dict with **messages**.

create_agent_basic.py

```
from langchain.agents import create_agent
agent = create_agent(
    model="google_genai:gemini-2.5-flash",          # string or a chat model instance
    tools=[get_weather, kb_search],
    system_prompt="You are a helpful HR + weather assistant. Use tools; cite sources.",
)
result = agent.invoke({"messages": [{"role": "user", "content": "How many sick leaves do I get?"}]})
print(result["messages"][-1].content)
for m in result["messages"]: m.pretty_print()            # Human → AI(tool_calls) → Tool → AI(final)
```

Parameter	Purpose
model	model id string or BaseChatModel instance
tools	list of tools (functions, @tool, BaseTool, dict schemas)
system_prompt	string or SystemMessage
response_format	Pydantic/TypedDict schema for a structured final answer (ToolStrategy or ProviderStrategy); result['structured_response']
state_schema	extend AgentState with custom keys
context_schema	typed runtime context (user_id, role…) passed via invoke(..., context=...)
checkpointer	short-term memory / persistence per thread_id (InMemorySaver, SqliteSaver…)
store	long-term memory store
middleware	list of middleware objects (below)
name, debug, cache, interrupt_before/after	misc / debugging

Structured final responses

agent_structured.py

```
from langchain.agents.structured_output import ToolStrategy, ProviderStrategy
class Answer(BaseModel): answer: str; sources: list[str]; confidence: float
agent = create_agent(model=llm, tools=[kb_search], response_format=ToolStrategy(Answer))   # ProviderStrategy uses native
        JSON schema
res = agent.invoke({"messages": [("user", "Sick leave policy?")]}); res["structured_response"]
```

Memory, context and streaming

agent_memory_stream.py

```
from langgraph.checkpoint.memory import InMemorySaver
from dataclasses import dataclass
@dataclass
class Ctx: user_id: str; role: str
agent = create_agent(model=llm, tools=tools, checkpointer=InMemorySaver(), context_schema=Ctx)
cfg = {"configurable": {"thread_id": "u42-chat1"}}
agent.invoke({"messages": [("user", "Hi, I am Mansavi")]}, cfg, context=Ctx("u42", "employee"))
agent.invoke({"messages": [("user", "What is my name?")]}, cfg, context=Ctx("u42", "employee"))   # remembers

for chunk in agent.stream({"messages": [("user", "Weather in Pune?")]}, cfg, stream_mode="updates"):
    print(chunk)                                    # {'model': {...}} then {'tools': {...}} ...
for token, meta in agent.stream({"messages": [("user", "...")]}, cfg, stream_mode="messages"):
    print(token.content, end="")                    # LLM tokens
```

10.4 Middleware — customising the agent loop

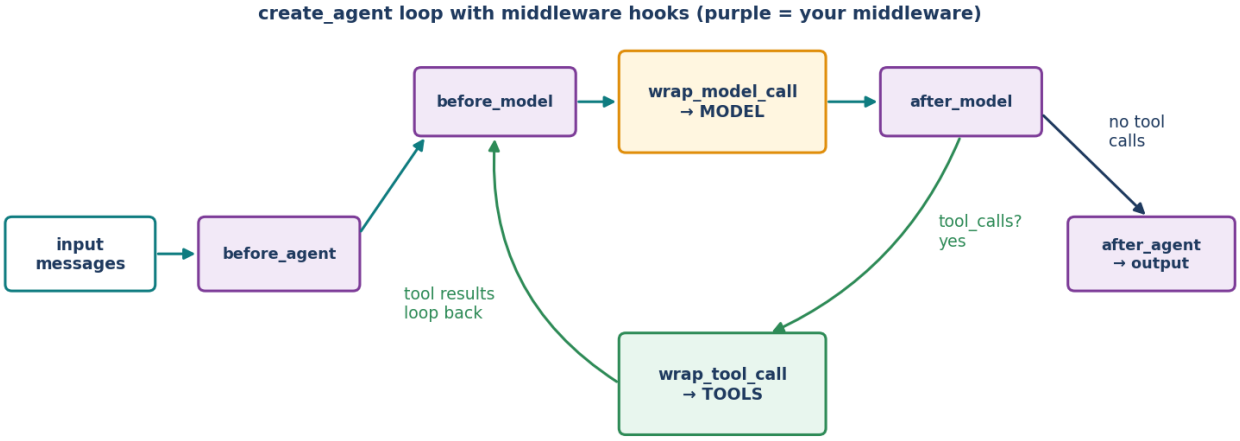


Figure: Where middleware hooks run inside the create_agent loop.

Middleware is the defining feature of create_agent: it controls dynamic prompts, summarisation, selective tool access, state management and guardrails — i.e. **context engineering**.[2]

Hook (decorator)	Runs	Typical use
@before_agent	once, before the agent starts	load user profile, validate input
@before_model	before every model call	trim/summarise messages, inject context
@after_model	after every model response	guardrails on output, stop conditions
@wrap_model_call	around the model call (can modify request or retry)	dynamic model selection, fallbacks, caching
@wrap_tool_call	around each tool execution	retries, permission checks, logging
@after_agent	once at the end	post-processing, saving memories
@dynamic_prompt	computes the system prompt per call	role-based prompts

These decorators and the AgentMiddleware base class are documented in the middleware reference.[4]

Prebuilt middleware

Middleware	What it does
SummarizationMiddleware	summarise old history when approaching a token limit
HumanInTheLoopMiddleware	pause for approve / edit / reject before sensitive tool calls
PIIMiddleware	detect and redact/mask/block PII (email, phone, cards, custom regex)
ModelCallLimitMiddleware / ToolCallLimitMiddleware	cap calls to control cost and loops
ModelFallbackMiddleware	fallback to alternative models on failure
ToolRetryMiddleware	retry failed tools with exponential backoff
LLMToolSelectorMiddleware	LLM pre-selects relevant tools when there are many
TodoListMiddleware	gives the agent a planning/to-do tool

Middleware	What it does
ContextEditingMiddleware	clear old tool outputs from context
LLMToolEmulator	simulate tools with an LLM for testing

Source: LangChain middleware reference.[4]

prebuilt_middleware.py

```
from langchain.agents.middleware import SummarizationMiddleware, HumanInTheLoopMiddleware, PIIMiddleware,
        ModelCallLimitMiddleware
agent = create_agent(
    model=llm, tools=[read_email, send_email], checkpointer=InMemorySaver(),
    middleware=[
        PIIMiddleware("email", strategy="redact", apply_to_input=True),
        SummarizationMiddleware(model=llm, trigger={"tokens": 4000}),
        ModelCallLimitMiddleware(run_limit=10),
        HumanInTheLoopMiddleware(interrupt_on={"send_email": {"allowed_decisions": ["approve", "edit", "reject"]}}),
    ])
from langgraph.types import Command
r = agent.invoke({"messages": [("user", "Reply to Rahul confirming Friday")]}, cfg)     # pauses → r["__interrupt__"]
agent.invoke(Command(resume={"decisions": [{"type": "approve"}]}), cfg)                  # resume after review
```

The exact resume payload format for HumanInTheLoopMiddleware can differ by version — check the current docs. The parameter names above follow the v1 release notes.[2]

Custom middleware

custom_middleware.py

```
from langchain.agents.middleware import before_model, wrap_model_call, dynamic_prompt, ModelRequest, AgentState
from langgraph.runtime import Runtime

@dynamic_prompt
def role_prompt(request: ModelRequest) -> str:
    role = request.runtime.context.role
    return "You are an HR assistant." + (" You may show salary bands." if role == "manager" else " Never reveal salaries.")

@wrap_model_call
def pick_model(request: ModelRequest, handler):
    if len(request.state["messages"]) > 20: request = request.override(model=big_llm)   # long chats → stronger model
    return handler(request)

@before_model
def block_after_n(state: AgentState, runtime: Runtime) -> dict | None:
    if len(state["messages"]) > 60: return {"messages": [AIMessage("Conversation too long, please start a new chat.")],
        "jump_to": "end"}
    return None

agent = create_agent(model=llm, tools=tools, context_schema=Ctx, middleware=[role_prompt, pick_model, block_after_n])
```

Class-based middleware: subclass **AgentMiddleware** and implement before_model / after_model / wrap_model_call / wrap_tool_call, optionally declaring extra state keys and tools. Middleware are applied in list order (before-hooks in order, after-hooks in reverse, wrap hooks nested).

CHAPTER 11

Callbacks, LangSmith, Debugging & Evaluation

Callback system • handlers & events • tracing with LangSmith • debugging • datasets & evaluators • testing LangChain code

11.1 Callbacks

Callbacks hook into every step (chain start/end, LLM start/new token/end, tool start/end, retriever start/end, errors) for logging, streaming, monitoring and cost tracking. Pass them in config['callbacks'] (propagates to children) or to a component's constructor (only that component).

callbacks.py

```
from langchain_core.callbacks import BaseCallbackHandler
class CostLogger(BaseCallbackHandler):
    def on_llm_end(self, response, **kw):
        u = response.generations[0][0].message.usage_metadata; print("tokens:", u)
    def on_tool_start(self, serialized, input_str, **kw): print("tool:", serialized.get("name"), input_str)
    def on_chain_error(self, error, **kw): print("ERROR", error)
chain.invoke(inputs, config={"callbacks": [CostLogger()]})
# Async apps: subclass AsyncCallbackHandler. Many events: on_chat_model_start, on_llm_new_token, on_retriever_end,
        on_agent_action...
```

11.2 LangSmith: tracing and evaluation

- Set LANGSMITH_TRACING=true and LANGSMITH_API_KEY → every chain/agent/graph run is traced automatically: inputs, outputs, latency, tokens, cost, errors per step.
- Use run_name, tags and metadata in config to filter traces (e.g. by user or version). Decorate plain functions with **@traceable** to trace non-LangChain code.
- **Datasets**: collections of (inputs, reference outputs); create from traces or CSV.
- **Evaluators**: custom Python functions, LLM-as-judge (correctness, groundedness, helpfulness), heuristic checks; run with evaluate().
- Compare experiments side by side, run online evaluation on production traces, collect user feedback, and manage prompts in the prompt hub.

langsmith_eval.py

```
from langsmith import Client, traceable
from langsmith.evaluation import evaluate
client = Client()
ds = client.create_dataset("hr-qa")
client.create_examples(dataset_id=ds.id, inputs=[{"question": "Sick leaves per year?"}], outputs=[{"answer": "12"}])
def correct(run, example) -> dict:
    return {"key": "contains_answer", "score": int(example.outputs["answer"] in run.outputs["output"])}
evaluate(lambda x: {"output": rag.invoke(x["question"])}, data="hr-qa", evaluators=[correct], experiment_prefix="rag-v2")
```

11.3 Debugging and testing

Tool	Use
set_debug(True) / set_verbose(True)	print all inputs/outputs
chain.get_graph().print_ascii()	see the structure
LangSmith traces	step-by-step inspection in production
Fake models: GenericFakeChatModel, FakeListChatModel (langchain_core.language_models.fake_chat_models)	deterministic unit tests without API calls
pytest + mocking tools	test routers, parsers and tools in isolation

test_chain.py

```
from langchain_core.language_models.fake_chat_models import GenericFakeChatModel
def test_chain():
    fake = GenericFakeChatModel(messages=iter([AIMessage("billing")]))
    chain = prompt | fake | StrOutputParser()
    assert chain.invoke({"ticket": "charged twice"}) == "billing"
```

CHAPTER 12

Production Patterns for LangChain Apps

Async & concurrency • serving with FastAPI • streaming endpoints • caching • rate limits • security • cost • MCP • configuration

12.1 Serving with FastAPI

serve.py

```
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
app = FastAPI()
class Q(BaseModel): question: str; session_id: str

@app.post("/ask")
async def ask(q: Q): return {"answer": await rag.ainvoke(q.question)}

@app.post("/ask/stream")
async def ask_stream(q: Q):
    async def gen():
        async for tok in rag.astream(q.question): yield f"data: {tok}\n\n"
    return StreamingResponse(gen(), media_type="text/event-stream")
```

LangServe (add_routes) was the old way to expose runnables as REST; for new projects use FastAPI directly or LangGraph Platform/Server for graphs.

12.2 Production checklist

Area	Practice
Async	use ainvoke/astream in web servers; abatch with max_concurrency
Reliability	timeouts, with_retry, with_fallbacks, ModelFallbackMiddleware
Cost	small models for routing/grading, caching (set_llm_cache), trimming/summarising history, max_tokens, call-limit middleware
Rate limits	InMemoryRateLimiter, queueing, backoff on 429
Security	keys in env/secret manager; treat retrieved/tool text as untrusted (prompt injection); least-privilege tools; HITL for writes; PII middleware
Observability	LangSmith tracing, tags/metadata per user and version, token logging
Quality	golden datasets + evaluators in CI; structured outputs
Config	configurable_fields/alternatives for model switching; versioned prompts

12.3 MCP tools

mcp_tools.py

```
from langchain_mcp_adapters.client import MultiServerMCPClient
client = MultiServerMCPClient({
    "math": {"command": "python", "args": ["math_server.py"], "transport": "stdio"},
    "weather": {"url": "http://localhost:8000/mcp", "transport": "streamable_http"}})
tools = await client.get_tools()
agent = create_agent(model=llm, tools=tools)
```

Part II — LangGraph

State, nodes & edges, reducers, Send & Command, workflow patterns, agents, persistence & time travel, memory, human-in-the-loop, streaming, subgraphs, multi-agent, Functional API, deployment & testing

CHAPTER 13

LangGraph Fundamentals: Mental Model

Why LangGraph • graphs vs chains vs agents • Pregel execution & super-steps • Graph API vs Functional API • first graph • compile & invoke • visualisation

13.1 Why LangGraph?

LangGraph is a low-level orchestration framework and runtime for building stateful, long-running agents and workflows. It does not hide prompts or architecture; instead it provides the infrastructure every serious agent needs: **durable execution** (resume after failures), **persistence/memory**, **human-in-the-loop**, **streaming**, and **arbitrary control flow** (loops, branches, parallelism, sub-graphs). LangChain's create_agent itself runs on LangGraph.[3]

Concept	Meaning
Workflow	predetermined code paths where LLMs fill in steps (prompt chaining, routing, parallelisation…)
Agent	the LLM dynamically directs its own process and tool use
Graph	nodes (units of work) connected by edges (control flow) operating on shared state
State	a typed snapshot of the application, the single source of truth passed between nodes
Super-step	one 'tick' of the graph: all nodes scheduled for that step run (in parallel), then their updates are applied

13.2 How a graph executes (Pregel model)

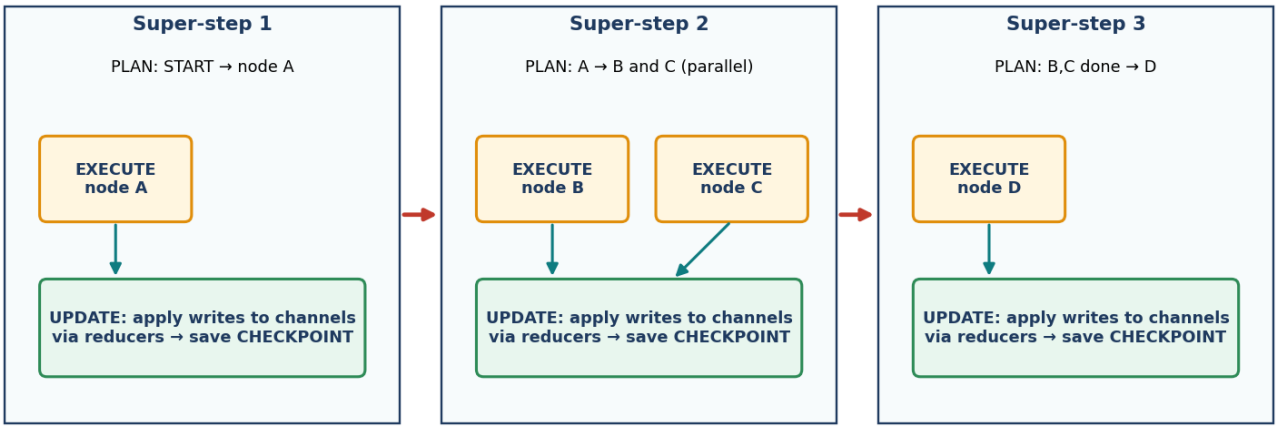


Figure: Super-steps: plan which nodes run → execute them (in parallel) → apply updates through reducers → checkpoint → repeat.

LangGraph's runtime is inspired by Google's **Pregel** (message passing / bulk synchronous parallel). Each state key is a **channel**. When a node finishes, its returned dict is written to channels; at the end of the super-step all writes are applied using each key's **reducer**, a **checkpoint** is saved, and nodes whose incoming edges were triggered become active for the next step. Execution ends when

no nodes are active (reached END) or the **recursion limit** (default 25 super-steps) is hit.

- Nodes in the same super-step run in parallel and see the **same** state snapshot.
- Updates from parallel nodes to the same key need a reducer (otherwise InvalidUpdateError).
- Super-steps are **transactional**: if one parallel node fails, none of that step's updates are applied (with a checkpointer, successful nodes' writes are saved as pending so they are not re-run on resume).

13.3 Your first graph

first_graph.py

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class State(TypedDict):              # 1) STATE schema
    text: str
    word_count: int
    summary: str

def count(state: State) -> dict:     # 2) NODES: read state, return a PARTIAL update
    return {"word_count": len(state["text"].split())}
def summarise(state: State) -> dict:
    return {"summary": llm.invoke(f"Summarise in one line: {state['text']}").content}

builder = StateGraph(State)          # 3) BUILD
builder.add_node("count", count)
builder.add_node("summarise", summarise)
builder.add_edge(START, "count")     # 4) EDGES
builder.add_edge("count", "summarise")
builder.add_edge("summarise", END)
graph = builder.compile()            # 5) COMPILE (validates; adds checkpointer/interrupts if given)

out = graph.invoke({"text": "LangGraph lets you build stateful agents as graphs."})   # 6) RUN
print(out)          # {'text': ..., 'word_count': 8, 'summary': '...'}  ← final full state
```

Step	API
Define state	TypedDict / Pydantic BaseModel / dataclass
Add nodes	builder.add_node(name, fn) or add_node(fn) (name = function name); add_sequence([f1, f2, f3])
Add edges	add_edge, add_conditional_edges, START/END constants (set_entry_point / set_finish_point are older equivalents)
Compile	builder.compile(checkpointer=None, store=None, interrupt_before=None, interrupt_after=None, cache=None, name=None)
Run	invoke / ainvoke / stream / astream / batch — a compiled graph is a Runnable

Visualising a graph

visualise.py

```
graph.get_graph().print_ascii()
print(graph.get_graph().draw_mermaid())                          # paste into mermaid.live
from IPython.display import Image; Image(graph.get_graph().draw_mermaid_png())   # in Jupyter
open("graph.png", "wb").write(graph.get_graph(xray=True).draw_mermaid_png())     # xray=True expands
        subgraphs
```

13.4 Graph API vs Functional API

	Graph API (StateGraph)	Functional API (@entrypoint, @task)
Style	explicit nodes and edges	ordinary Python control flow (if/for/while)
State	shared typed state + reducers	function-scoped variables; return values
Visualisation	yes	limited
Gets persistence, streaming, HITL?	yes	yes
Best for	complex branching, multi-agent, teams that want a visible structure	adding durability/HITL to existing code with minimal changes

CHAPTER 14

State, Nodes and Edges in Depth

State schemas • reducers • MessagesState & add_messages • input/output/private schemas • nodes (signature, config, runtime) • normal & conditional edges • parallel fan-out/fan-in • Send (map-reduce) • Command (update + goto) • deferred nodes • recursion limit

14.1 State schemas

state_schemas.py

```
from typing import TypedDict, Annotated, Literal
from pydantic import BaseModel, Field
from dataclasses import dataclass, field

class S1(TypedDict):                # most common; no runtime validation
    question: str; docs: list; answer: str

class S2(BaseModel):                # runtime validation of node INPUTS; slower; defaults supported
    question: str; retries: int = Field(0, ge=0)

@dataclass
class S3:                           # defaults supported
    question: str; docs: list = field(default_factory=list)
```

Nodes return **partial updates** (only keys they change). You never mutate state in place — you return a dict and the runtime merges it. Keys not returned are untouched.

14.2 Reducers

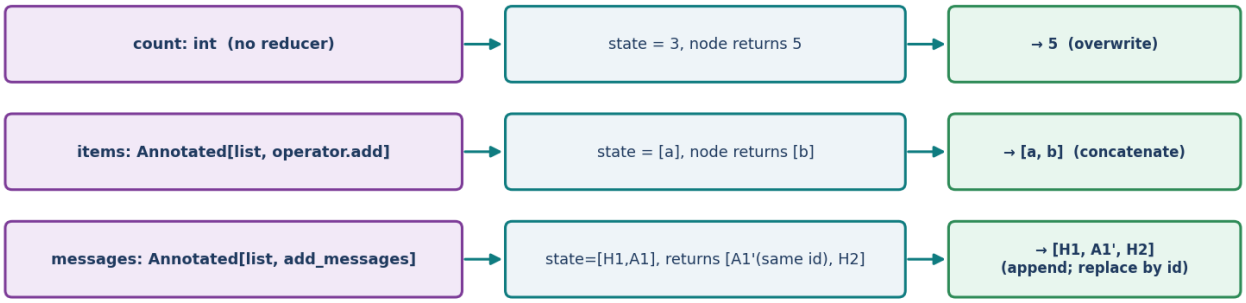


Figure: Reducers decide how an update merges into the existing value.

reducers.py

```
import operator
from langgraph.graph.message import add_messages
class State(TypedDict):
    count: int                                      # overwrite
    logs: Annotated[list[str], operator.add]        # concatenate lists (needed for parallel writers)
    messages: Annotated[list, add_messages]         # append messages, replace by id, delete via RemoveMessage
    total: Annotated[int, operator.add]             # sum
def keep_max(old, new): return max(old, new)
class S(TypedDict): best_score: Annotated[float, keep_max]   # custom reducer (old, new) -> merged
```

add_messages and MessagesState

- Appends new messages; if a message with the same **id** exists it is **replaced** (used to edit messages).
- Accepts dicts/tuples ('user', 'hi') and converts them to message objects.
- Returning **RemoveMessage(id=...)** deletes a message; RemoveMessage(id=REMOVE_ALL_MESSAGES) clears all.
- **MessagesState** is a prebuilt TypedDict with messages: Annotated[list[AnyMessage], add_messages]; subclass it to add keys.

messages_state.py

```
from langgraph.graph import MessagesState
from langchain_core.messages import RemoveMessage
class AgentState(MessagesState):
    user_id: str
    summary: str
def delete_old(state: AgentState):
    return {"messages": [RemoveMessage(id=m.id) for m in state["messages"][:-6]]}   # keep last 6

from langgraph.types import Overwrite
def reset_logs(state): return {"logs": Overwrite([])}      # bypass the reducer and replace the value
```

Input, output and private state schemas

io_schemas.py

```
class InputState(TypedDict): question: str
class OutputState(TypedDict): answer: str
class OverallState(InputState, OutputState): docs: list; draft: str          # internal keys
class Private(TypedDict): scratch: str                                       # used only between two nodes
builder = StateGraph(OverallState, input_schema=InputState, output_schema=OutputState)
def retrieve(state: InputState) -> Private: return {"scratch": "..."}         # a node can read/write other schemas
graph = builder.compile(); graph.invoke({"question": "q"})                   # returns only {'answer': ...}
```

14.3 Nodes

nodes.py

```
from langchain_core.runnables import RunnableConfig
from langgraph.runtime import Runtime

def plain(state: State) -> dict: ...                                    # state only
def with_config(state: State, config: RunnableConfig) -> dict:          # thread_id, tags, configurable values
    tid = config["configurable"]["thread_id"]; ...
@dataclass
class Ctx: user_id: str; model_name: str = "gemini-2.5-flash"
def with_runtime(state: State, runtime: Runtime[Ctx]) -> dict:          # runtime.context, runtime.store,
        runtime.stream_writer
    uid = runtime.context.user_id; mems = runtime.store.search(("users", uid)); ...
builder = StateGraph(State, context_schema=Ctx)
graph.invoke({"question": "..."}, context=Ctx(user_id="u42"))

async def anode(state): return {"answer": (await llm.ainvoke(state["question"])).content}   # async nodes: use
        ainvoke/astream on graph
```

Node feature	How
Retry policy	add_node('call_api', fn, retry_policy=RetryPolicy(max_attempts=3, retry_on=TimeoutError))
Node caching	add_node('expensive', fn, cache_policy=CachePolicy(ttl=300)); compile(cache=InMemoryCache())
Deferred node	add_node('aggregate', fn, defer=True) — waits until all pending parallel branches finish (fan-in with unequal path lengths)

Node feature	How
Any runnable as node	add_node('rag', rag_chain) — a chain or even a compiled subgraph
Node returns Command	update state AND choose the next node (below)

14.4 Edges

Normal and conditional edges

edges.py

```
builder.add_edge("retrieve", "grade")                          # always

def route_after_grade(state: State) -> Literal["generate", "rewrite"]:     # router function reads state, returns node
        name(s)
    return "generate" if state["docs"] else "rewrite"
builder.add_conditional_edges("grade", route_after_grade)                  # Literal return type lets the graph draw the
        edges

# with an explicit path map (router returns a key → mapped to node)
builder.add_conditional_edges("agent", lambda s: "tools" if s["messages"][-1].tool_calls else "done",
                              {"tools": "tools", "done": END})
builder.add_conditional_edges(START, classify_route)                       # conditional entry point
```

Parallel fan-out and fan-in

parallel.py

```
class State(TypedDict): topic: str; results: Annotated[list, operator.add]       # reducer REQUIRED for parallel writes
builder.add_edge(START, "web"); builder.add_edge(START, "kb"); builder.add_edge(START, "news")   # fan-out: same
        super-step
builder.add_edge(["web", "kb", "news"], "combine")                                  # fan-in: wait for ALL three
builder.add_edge("combine", END)
```

Send — dynamic map-reduce

When the number of parallel branches is only known at runtime (e.g. one branch per sub-question or per document), a conditional edge can return a list of **Send(node_name, custom_state)** objects. Each Send runs the target node once with its own input, in parallel.

send_map_reduce.py

```
from langgraph.types import Send
class Overall(TypedDict): topic: str; subjects: list[str]; jokes: Annotated[list, operator.add]; best: str
class JokeIn(TypedDict): subject: str

def plan(state: Overall): return {"subjects": llm.with_structured_output(Subjects).invoke(state["topic"]).items}
def fan_out(state: Overall): return [Send("write_joke", {"subject": s}) for s in state["subjects"]]   # MAP
def write_joke(state: JokeIn): return {"jokes": [llm.invoke(f"Joke about {state['subject']}").content]}
def pick_best(state: Overall): return {"best": llm.invoke(f"Pick the best: {state['jokes']}").content}   # REDUCE

b = StateGraph(Overall)
b.add_node("plan", plan); b.add_node("write_joke", write_joke); b.add_node("pick_best", pick_best)
b.add_edge(START, "plan"); b.add_conditional_edges("plan", fan_out, ["write_joke"])
b.add_edge("write_joke", "pick_best"); b.add_edge("pick_best", END)
```

Command — update state and route in one return

command.py

```
from langgraph.types import Command
def triage(state: State) -> Command[Literal["billing", "tech", "__end__"]]:
    label = classifier.invoke(state["messages"]).label
    return Command(update={"category": label}, goto=label if label in ("billing", "tech") else END)
builder.add_node("triage", triage)          # no conditional edge needed — the type hint tells the graph possible targets
# Command(goto=..., graph=Command.PARENT) navigates to a node in the PARENT graph (multi-agent handoffs from a subgraph)
# Tools can also return Command(update={...}) to change graph state.
```

Use conditional edges when	Use Command when
routing depends only on state and you want routing logic separate from node logic	a node must both update state and decide where to go (handoffs, agent-to-agent transfer)

14.5 Recursion limit and loops

recursion.py

```
from langgraph.errors import GraphRecursionError
try:
    graph.invoke(inputs, {"recursion_limit": 15})          # max super-steps (default 25)
except GraphRecursionError:
    ...                                                    # return a graceful message / escalate
# Better: track your own counter in state and route to END after N retries
class S(TypedDict): retries: int
def should_retry(s): return "rewrite" if s["retries"] < 2 else "give_up"
```

CHAPTER 15

Workflow Patterns & Building Agents in LangGraph

Prompt chaining • routing • parallelisation • orchestrator–workers • evaluator–optimizer • ReAct agent from scratch • ToolNode & tools_condition • prebuilt agents • structured output in graphs • agentic RAG / CRAG / Self-RAG graph

15.1 Six core patterns

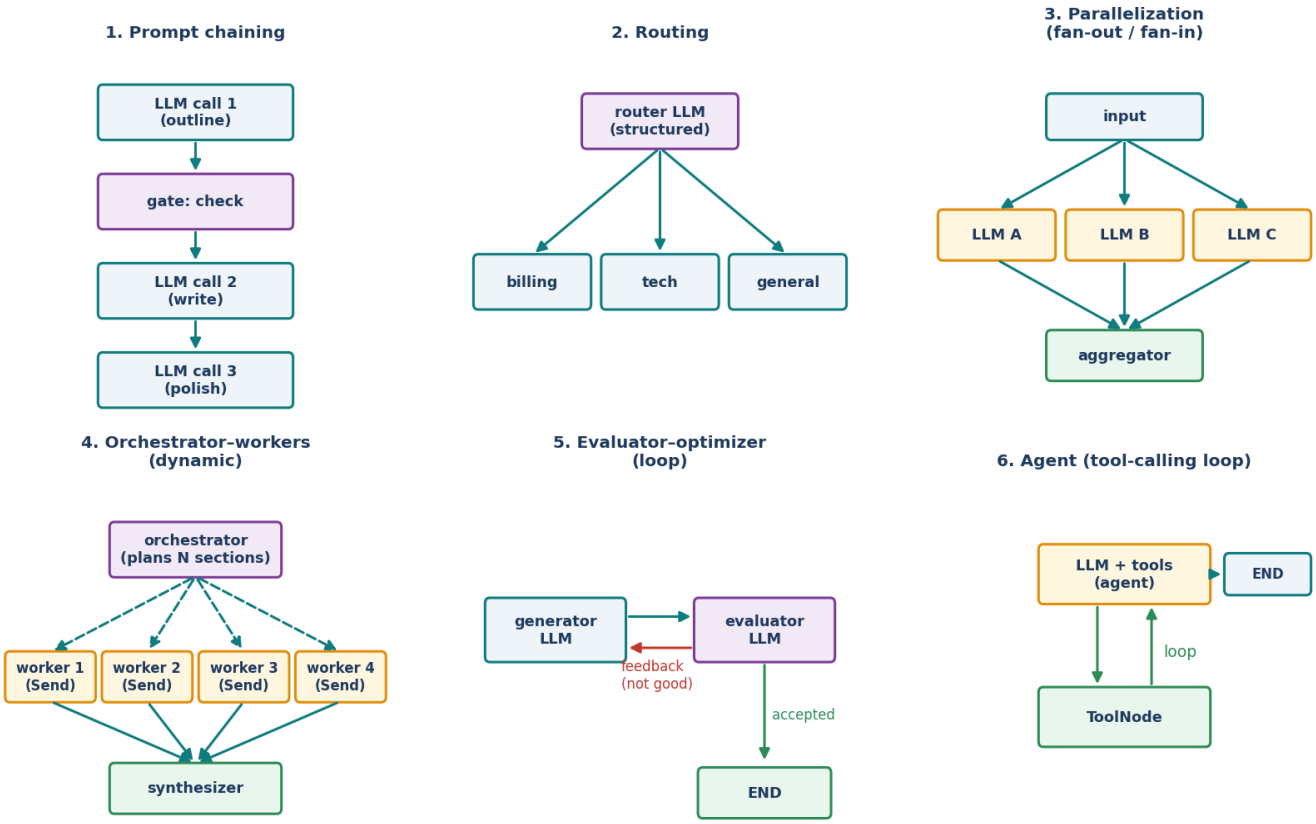


Figure: Workflow and agent patterns (workflows = fixed paths; agents = model-driven loops).

Pattern	Graph building blocks	Example
Prompt chaining	sequential edges + a gate (conditional edge) that can stop early	outline → check → write → polish
Routing	classifier node with structured output + conditional edges	ticket triage to billing/tech/general
Parallelisation	fan-out edges + reducer + fan-in	check guardrails while answering; multiple reviewers voting
Orchestrator–workers	planner + Send to workers + reducer + synthesizer	report writer with one worker per section
Evaluator–optimizer	generator ⇄ evaluator loop with a conditional edge	translation/code refined until it passes

Pattern	Graph building blocks	Example
Agent	LLM with bound tools ⇄ ToolNode loop	assistant using search, DB and APIs

Routing example

routing_graph.py

```
class Route(BaseModel): step: Literal["poem", "story", "joke"]
router_llm = llm.with_structured_output(Route)
class S(TypedDict): input: str; decision: str; output: str
def route(s): return {"decision": router_llm.invoke(s["input"]).step}
def poem(s): return {"output": llm.invoke(f"Poem: {s['input']}").content}
def story(s): return {"output": llm.invoke(f"Story: {s['input']}").content}
def joke(s): return {"output": llm.invoke(f"Joke: {s['input']}").content}
b = StateGraph(S)
for n, f in [("route", route), ("poem", poem), ("story", story), ("joke", joke)]: b.add_node(n, f)
b.add_edge(START, "route"); b.add_conditional_edges("route", lambda s: s["decision"], ["poem", "story",
        "joke"])
for n in ["poem", "story", "joke"]: b.add_edge(n, END)
```

Evaluator-optimizer example

evaluator_optimizer.py

```
class Feedback(BaseModel): grade: Literal["pass", "fail"]; feedback: str
class S(TypedDict): topic: str; draft: str; feedback: str; grade: str; rounds: int
def generate(s):
    extra = f" Improve using feedback: {s['feedback']}" if s.get("feedback") else ""
    return {"draft": llm.invoke(f"Write a tagline for {s['topic']}.{extra}").content, "rounds": s.get("rounds", 0) + 1}
def evaluate(s):
    f = llm.with_structured_output(Feedback).invoke(f"Grade this tagline: {s['draft']}"); return {"grade": f.grade,
        "feedback": f.feedback}
b = StateGraph(S); b.add_node("generate", generate); b.add_node("evaluate", evaluate)
b.add_edge(START, "generate"); b.add_edge("generate", "evaluate")
b.add_conditional_edges("evaluate", lambda s: END if s["grade"] == "pass" or s["rounds"] >= 3 else "generate")
```

15.2 A ReAct agent from scratch

react_from_scratch.py

```
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.prebuilt import ToolNode, tools_condition
tools = [get_weather, kb_search]
model = llm.bind_tools(tools)

def call_model(state: MessagesState):
    sys = SystemMessage("You are a helpful assistant. Use tools when needed.")
    return {"messages": [model.invoke([sys] + state["messages"])]}

def should_continue(state: MessagesState) -> Literal["tools", "__end__"]:      # = what tools_condition does
    return "tools" if state["messages"][-1].tool_calls else END

b = StateGraph(MessagesState)
b.add_node("agent", call_model)
b.add_node("tools", ToolNode(tools))          # executes all tool_calls of the last AIMessage (in parallel) → ToolMessages
b.add_edge(START, "agent")
b.add_conditional_edges("agent", should_continue)     # or tools_condition
b.add_edge("tools", "agent")                  # loop
agent = b.compile(checkpointer=InMemorySaver())
agent.invoke({"messages": [("user", "Weather in Pune, and our WFH policy?")]}, {"configurable": {"thread_id": "1"}})
```

Prebuilt	What it does
ToolNode(tools, handle_tool_errors=True)	runs tool calls from the last AIMessage; returns ToolMessages; errors become messages so the model can recover
tools_condition	router: 'tools' if last message has tool_calls else END
create_react_agent (langgraph.prebuilt)	prebuilt ReAct graph — deprecated in v1 in favour of langchain.agents.create_agent

Prebuilt	What it does
InjectedState / InjectedStore	annotations to inject graph state/store into tool arguments

Since v1, LangGraph's create_react_agent is deprecated in favour of LangChain's create_agent.[3] Build the graph by hand when you need custom nodes (e.g. a grader, a planner, approvals) between model and tools.

15.3 Agentic RAG as a graph (CRAG + Self-RAG style)

crag_self_rag_graph.py

```
class RAGState(TypedDict):
    question: str; docs: list; answer: str; web_used: bool; retries: int
class Grade(BaseModel): relevant: Literal["yes", "no"]
class Check(BaseModel): grounded: Literal["yes", "no"]

def retrieve(s): return {"docs": retriever.invoke(s["question"])}
def grade_docs(s):
    g = llm.with_structured_output(Grade)
    keep = [d for d in s["docs"] if g.invoke(f"Q: {s['question']}\nDoc: {d.page_content}").relevant == "yes"]
    return {"docs": keep}
def rewrite(s): return {"question": llm.invoke(f"Rewrite for better search: {s['question']}").content, "retries":
        s.get("retries", 0) + 1}
def web_search(s): return {"docs": s["docs"] + [Document(page_content=r["content"]) for r in tavily.invoke(s["question"])],
        "web_used": True}
def generate(s): return {"answer": rag_prompt_chain.invoke({"context": format_docs(s["docs"]), "question": s["question"]})}
def route_docs(s): return "generate" if s["docs"] else ("rewrite" if s.get("retries", 0) < 1 else "web_search")
def check_grounded(s):
    ok = llm.with_structured_output(Check).invoke(f"Docs: {format_docs(s['docs'])}\nAnswer: {s['answer']}").grounded ==
        "yes"
    return END if ok or s.get("retries", 0) >= 2 else "rewrite"

b = StateGraph(RAGState)
for n, f in [("retrieve", retrieve), ("grade_docs", grade_docs), ("rewrite", rewrite), ("web_search", web_search),
        ("generate", generate)]: b.add_node(n, f)
b.add_edge(START, "retrieve"); b.add_edge("retrieve", "grade_docs")
b.add_conditional_edges("grade_docs", route_docs, ["generate", "rewrite", "web_search"])
b.add_edge("rewrite", "retrieve"); b.add_edge("web_search", "generate")
b.add_conditional_edges("generate", check_grounded, ["rewrite", END])
crag = b.compile()
```

CHAPTER 16

Persistence, Threads, Checkpoints & Time Travel

Checkpointers • threads • StateSnapshot • get_state / get_state_history • update_state • replay & fork • durability modes • serialisation & encryption • pending writes & fault tolerance

16.1 Checkpointers and threads

Compiling with a **checkpointer** saves a **checkpoint** (snapshot of state) at every super-step. Checkpoints are grouped into **threads**, identified by config['configurable']['thread_id'] — one thread per conversation/session/job. This single feature enables memory, human-in-the-loop, time travel and fault tolerance.

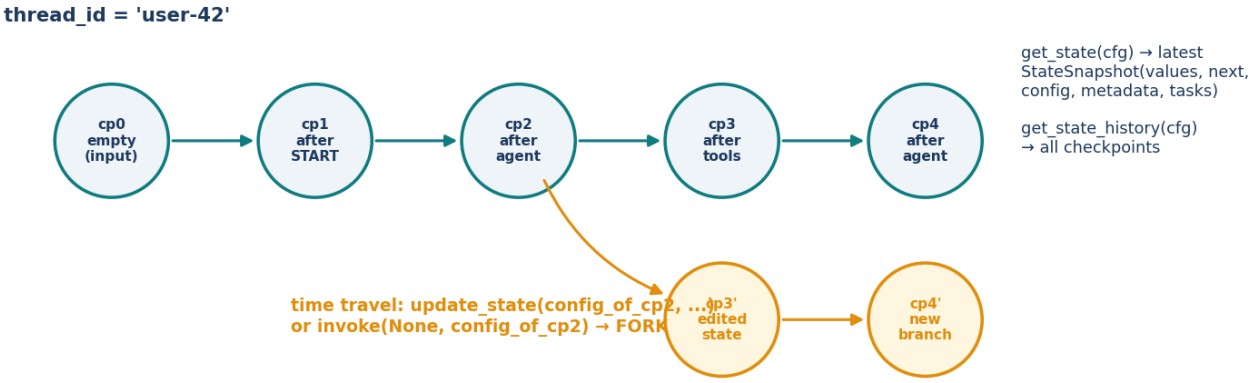


Figure: A thread is a chain of checkpoints; you can inspect, edit and fork from any of them.

Checkpointer	Package	Use
InMemorySaver (MemorySaver)	langgraph	dev/tests; lost on restart
SqliteSaver / AsyncSqliteSaver	langgraph-checkpoint-sqlite	local apps, prototypes
PostgresSaver / AsyncPostgresSaver	langgraph-checkpoint-postgres	production (call .setup() once to create tables)
Others	Redis, MongoDB, cloud community savers	production options
LangGraph Platform	managed	persistence handled automatically

checkpointers.py

```
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.checkpoint.sqlite import SqliteSaver
import sqlite3
graph = builder.compile(checkpointer=InMemorySaver())
saver = SqliteSaver(sqlite3.connect("ckpt.db", check_same_thread=False)); graph = builder.compile(checkpointer=saver)
# with SqliteSaver.from_conn_string("ckpt.db") as saver: ...     (context-manager form)
# from langgraph.checkpoint.postgres import PostgresSaver
# with PostgresSaver.from_conn_string(DB_URI) as saver: saver.setup(); graph = builder.compile(checkpointer=saver)

cfg = {"configurable": {"thread_id": "user42-session1"}}
graph.invoke({"messages": [("user", "Hi, I'm Mansavi")]}, cfg)
graph.invoke({"messages": [("user", "What's my name?")]}, cfg)        # state (messages) continues from the checkpoint
```

Watch out: Without a checkpointer, every invoke starts from empty state and interrupts/HITL cannot work. With one, you MUST pass a thread_id in config. Common bug: forgetting thread_id → 'Checkpointer requires one or more of the following configurable keys'.

16.2 Inspecting and modifying state

inspect_state.py

```
snap = graph.get_state(cfg)              # StateSnapshot
snap.values                              # current state dict
snap.next                                # tuple of nodes that will run next (empty = finished; non-empty =
        interrupted/paused)
snap.config                              # includes checkpoint_id
snap.metadata                            # source ('input'/'loop'/'update'), step number, writes
snap.tasks                               # pending tasks, interrupts, errors

history = list(graph.get_state_history(cfg))       # newest first
before_tools = next(h for h in history if h.next == ("tools",))

graph.update_state(cfg, {"messages": [AIMessage("Edited answer")]})              # applies via reducers, creates a new
        checkpoint
graph.update_state(cfg, {"approved": True}, as_node="review")                   # pretend the update came from node
        'review' → next edges follow it
```

16.3 Time travel: replay and fork

time_travel.py

```
# REPLAY: resume execution from an earlier checkpoint (steps before it are not re-executed)
graph.invoke(None, before_tools.config)              # input None = continue from that checkpoint

# FORK: edit an earlier state, then continue — creates a new branch in the same thread
new_cfg = graph.update_state(before_tools.config, {"messages": [AIMessage(content="", tool_calls=[{"name": "get_weather",
        "args": {"city": "Delhi"}, "id": before_tools.values["messages"][-1].tool_calls[0]["id"]}])]})
graph.invoke(None, new_cfg)
```

Uses: debugging ('what if the model had chosen another tool?'), recovering from a bad step, letting a human correct the agent, and A/B exploring alternatives.

16.4 Durability and fault tolerance

Durability mode (graph.invoke(..., durability=...))	Behaviour
'exit'	persist only when the run finishes/interrupts — fastest, least safe
'async'	persist asynchronously while the next step runs (good default balance)
'sync'	persist before starting the next step — safest

- If a node fails mid-run, successful parallel nodes' writes are stored as **pending writes**; resuming (invoke(None, cfg)) re-runs only what failed.
- Make nodes **idempotent** (safe to re-run) and wrap side effects (payments, emails) in tasks or check-before-write logic, because a resumed node restarts from its beginning.
- Checkpoint data is serialised (JsonPlusSerializer); encryption at rest is available via an encrypted serializer.
- Long-running jobs: the thread survives process restarts — resume tomorrow with the same thread_id.

CHAPTER 17

Memory: Short-Term and Long-Term

Short-term (thread) memory • managing long conversations (trim, summarise, delete) • long-term memory with the Store • namespaces • semantic search • memory types (semantic, episodic, procedural) • writing memories (hot path vs background)

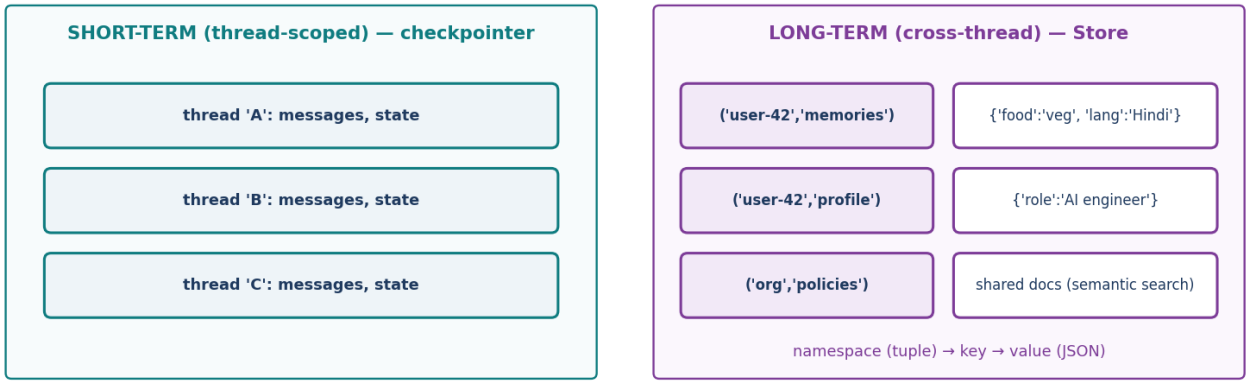


Figure: Short-term memory lives in the thread's checkpoints; long-term memory lives in a Store shared across threads.

17.1 Short-term memory

Short-term memory = the state of one thread (usually the message list) persisted by the checkpointer. Problem: conversations grow beyond the context window and cost. Strategies:

short_term.py

```
from langchain_core.messages import trim_messages, RemoveMessage
# (1) TRIM only what you send to the model (state keeps everything)
def call_model(state: MessagesState):
    msgs = trim_messages(state["messages"], strategy="last", max_tokens=2000, token_counter=llm, start_on="human",
        include_system=True)
    return {"messages": [model.invoke(msgs)]}

# (2) DELETE old messages from state permanently
def prune(state): return {"messages": [RemoveMessage(id=m.id) for m in state["messages"][:-10]]}

# (3) SUMMARISE older messages into a running summary
class S(MessagesState): summary: str
def summarise(state: S):
    prev = state.get("summary", "")
    prompt = f"Current summary: {prev}\nExtend it with the new messages above." if prev else "Summarise the conversation
        above."
    new = llm.invoke(state["messages"] + [HumanMessage(prompt)]).content
    return {"summary": new, "messages": [RemoveMessage(id=m.id) for m in state["messages"][:-2]]}
def maybe_summarise(state: S): return "summarise" if len(state["messages"]) > 12 else END
```

In create_agent, the same is done with **SummarizationMiddleware** or a before_model middleware that trims.

17.2 Long-term memory with the Store

A **Store** holds JSON documents across threads, organised by **namespace** (a tuple like ('users', user_id, 'memories')) and **key**. Pass it at compile time; nodes access it via runtime.store (or an injected store argument).

long_term_store.py

```
from langgraph.store.memory import InMemoryStore
store = InMemoryStore(index={"embed": emb, "dims": 384, "fields": ["text"]})     # index → enables semantic
        search
ns = ("users", "u42", "memories")
store.put(ns, "pref-1", {"text": "Prefers answers in Hindi"})
store.get(ns, "pref-1").value
store.search(ns, query="language preference", limit=3)        # semantic search (or filter=..., no query)
store.delete(ns, "pref-1")

def call_model(state: MessagesState, runtime: Runtime[Ctx]):
    ns = ("users", runtime.context.user_id, "memories")
    mems = runtime.store.search(ns, query=state["messages"][-1].content, limit=3)
    sys = "Known about the user: " + "; ".join(m.value["text"] for m in mems)
    resp = model.invoke([SystemMessage(sys)] + state["messages"])
    if "remember" in state["messages"][-1].content.lower():                  # write memory (hot path)
        runtime.store.put(ns, str(uuid4()), {"text": state["messages"][-1].content})
    return {"messages": [resp]}
graph = builder.compile(checkpointer=InMemorySaver(), store=store)          # production: PostgresStore
```

Memory type	Stores	Example
Semantic	facts	user's role, preferences (profile doc or collection of facts)
Episodic	past experiences	successful past trajectories used as few-shot examples
Procedural	rules / instructions	system prompt the agent updates from feedback

When to write memories	Pros / cons
Hot path (during the conversation, e.g. a save_memory tool)	immediately available; adds latency and complexity to the agent
Background (separate job after the conversation)	no latency; memories available later; needs scheduling (LangMem library helps)

CHAPTER 18

Human-in-the-Loop

interrupt() & Command(resume) • approve / reject • edit state • review tool calls • validate human input • multiple interrupts • static breakpoints • rules & pitfalls

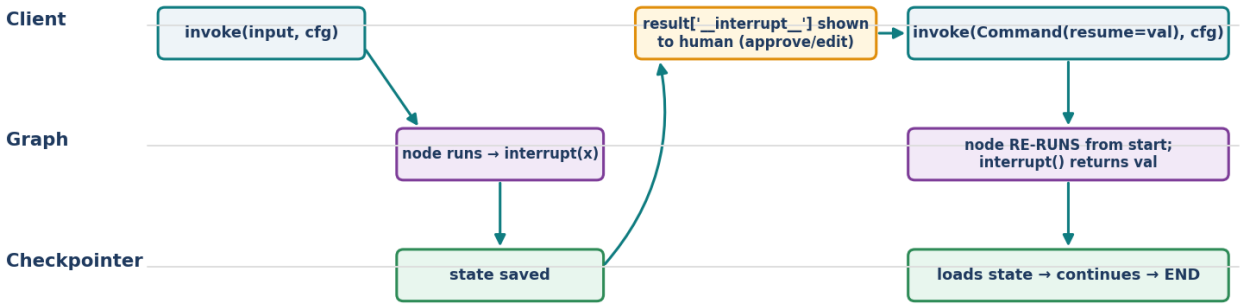


Figure: Interrupt lifecycle: pause and persist, show to a human, resume with their decision.

18.1 Dynamic interrupts

interrupt.py

```
from langgraph.types import interrupt, Command
class S(TypedDict): draft: str; status: str
def human_review(state: S) -> Command[Literal["publish", "rewrite"]]:
    decision = interrupt({"question": "Approve this draft?", "draft": state["draft"]})   # PAUSES here; payload goes to the
        client
    if decision["action"] == "approve": return Command(goto="publish")
    if decision["action"] == "edit": return Command(update={"draft": decision["text"]}, goto="publish")
    return Command(update={"status": "rejected"}, goto="rewrite")

graph = builder.compile(checkpointer=InMemorySaver())          # checkpointer REQUIRED
cfg = {"configurable": {"thread_id": "doc-7"}}
res = graph.invoke({"draft": "", "status": ""}, cfg)
print(res["__interrupt__"])                                    # [Interrupt(value={'question':..., 'draft':...}, id=...)]
graph.invoke(Command(resume={"action": "edit", "text": "Final copy"}), cfg)       # resume value becomes interrupt()'s
        return
```

18.2 Common HITL patterns

Pattern	Implementation
Approve / reject an action	interrupt before executing a risky tool; route on the decision
Edit state	human returns corrected values; Command(update=...)
Review tool calls	interrupt in a node between model and ToolNode (or inside the tool); allow approve / edit args / reject with feedback
Validate human input	loop: call interrupt() repeatedly until the answer is valid
Ask for missing info	agent calls an ask_human tool that internally calls interrupt()
Static breakpoints (debugging)	compile(interrupt_before=['tools']) or interrupt_after=[...]; resume with graph.invoke(None, cfg)

hitl_tool.py

```
@tool
def book_flight(origin: str, dest: str, date: str) -> str:
    """Book a flight."""
    resp = interrupt({"action": "book_flight", "args": {"origin": origin, "dest": dest, "date": date}})
    if resp["type"] == "accept": return do_booking(origin, dest, date)
    if resp["type"] == "edit": return do_booking(**resp["args"])
    return f"User declined booking. Reason: {resp.get('feedback', '')}"
```

18.3 Rules you must know (frequent interview / debugging topics)

1. On resume, the **whole node re-runs from its start**; interrupt() then returns the resume value instead of pausing. So code before interrupt() runs twice — keep it side-effect-free or idempotent.
2. Do not wrap interrupt() in a bare try/except — it works by raising a special exception (GraphInterrupt).
3. Multiple interrupt() calls in one node are matched to resume values **by order**; keep their order deterministic.
4. Interrupts in parallel branches: resume several at once with Command(resume={interrupt_id: value, ...}).
5. Payloads must be JSON-serialisable.
6. A checkpointer and a thread_id are mandatory.
7. With create_agent, HumanInTheLoopMiddleware packages the tool-approval pattern for you.

CHAPTER 19

Streaming

Stream modes: values, updates, messages, custom, debug • multiple modes • streaming LLM tokens with metadata • custom progress events • subgraph streaming • astream_events • streaming to a web client

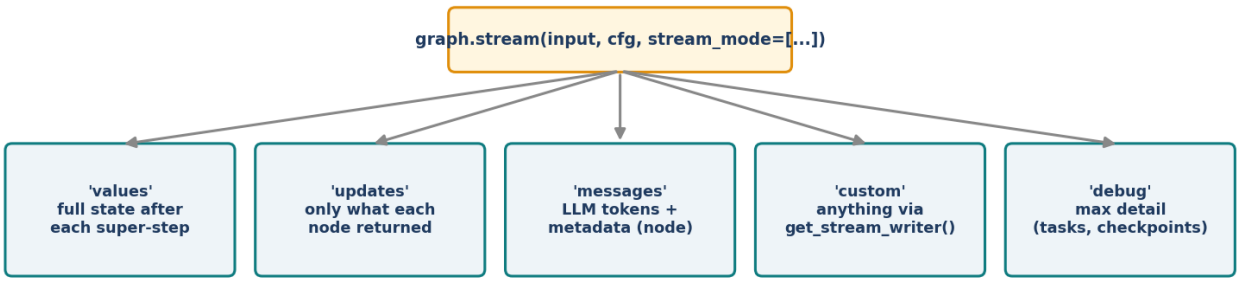


Figure: What each stream mode emits.

streaming.py

```
for chunk in graph.stream(inputs, cfg, stream_mode="updates"):
    print(chunk)                                    # {'agent': {'messages': [...]}} — per node

for state in graph.stream(inputs, cfg, stream_mode="values"):
    state["messages"][-1].pretty_print()            # full state after each step

for msg_chunk, meta in graph.stream(inputs, cfg, stream_mode="messages"):
    if meta["langgraph_node"] == "agent" and msg_chunk.content:   # filter tokens by node (or by tags)
        print(msg_chunk.content, end="", flush=True)

from langgraph.config import get_stream_writer
def retrieve(state):
    writer = get_stream_writer(); writer({"progress": "searching knowledge base..."})   # custom events
    docs = retriever.invoke(state["question"]); writer({"progress": f"found {len(docs)} docs"})
    return {"docs": docs}
for mode, chunk in graph.stream(inputs, cfg, stream_mode=["updates", "custom", "messages"]):   # several modes → (mode,
        chunk)
    ...
for ns, chunk in graph.stream(inputs, cfg, stream_mode="updates", subgraphs=True): ...       # include subgraph outputs; ns
        = path
```

Mode	Emits	Use
values	complete state after each super-step	simple UIs, debugging
updates	{node: update} per node	progress / step display
messages	(message_chunk, metadata) tuples from any LLM call inside nodes	token streaming for chat UIs
custom	data from get_stream_writer()	progress bars, tool status
debug / checkpoints / tasks	detailed execution events	deep debugging

For ChatGPT-like UIs combine **messages** (tokens) with **updates** or **custom** (step status), and send them as Server-Sent Events from FastAPI (see Chapter 12). Async: *async for ... in graph.astream(...)*.

CHAPTER 20

Subgraphs & Multi-Agent Systems

Subgraphs (shared vs different schemas) • subgraph persistence & state • multi-agent architectures • supervisor • swarm / handoffs • hierarchical teams • agents as tools • Command.PARENT • prebuilt libraries

20.1 Subgraphs

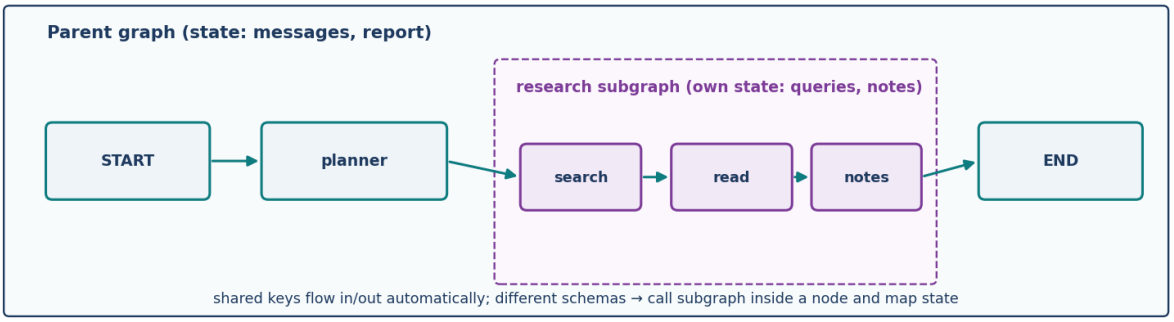


Figure: A compiled graph used inside another graph.

subgraphs.py

```
# (A) Shared state keys → add the compiled subgraph directly as a node
class Shared(TypedDict): messages: Annotated[list, add_messages]; report: str
research = research_builder.compile()              # its state also has 'messages' (+ private keys)
parent = StateGraph(Shared); parent.add_node("research", research); ...

# (B) Different schemas → call the subgraph inside a node and transform state
def call_research(state: ParentState) -> dict:
    out = research.invoke({"query": state["topic"]})          # map parent → child
    return {"report": out["notes"]}                           # map child → parent
parent.add_node("research", call_research)
```

- Persistence: give the checkpointer only to the parent; subgraphs inherit it. Use compile(checkpointer=True) on a subgraph if it needs its own memory across calls.
- Inspect subgraph state with graph.get_state(cfg, subgraphs=True); stream with subgraphs=True; visualise with get_graph(xray=True).
- Interrupts inside subgraphs work and bubble up to the parent.

20.2 Multi-agent architectures

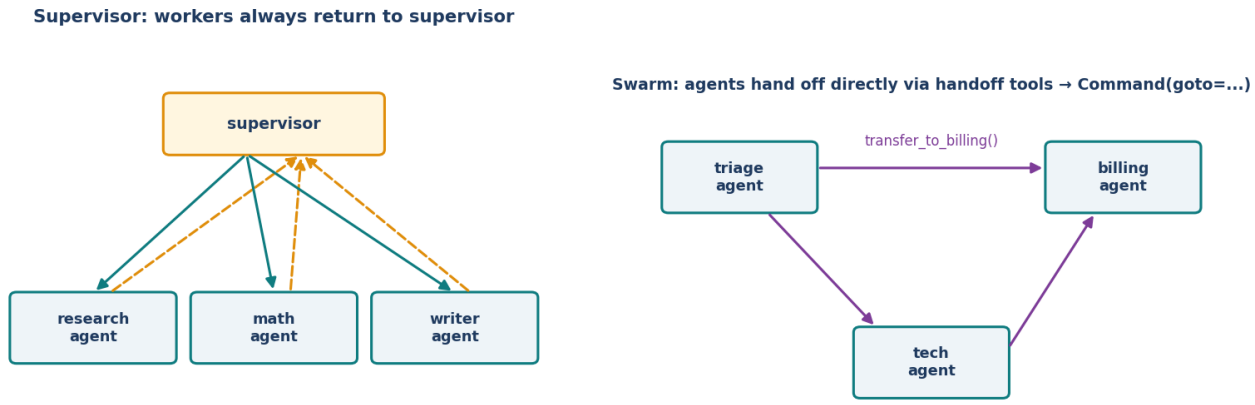


Figure: Supervisor (centralised) vs swarm (decentralised handoffs).

Architecture	How	Pros / cons
Single agent + many tools	one LLM chooses among tools	simplest; degrades with too many tools or domains
Supervisor	a supervisor LLM routes to worker agents (via structured output or handoff tools); workers return to it	clear control, easy to debug; supervisor is a bottleneck / extra LLM calls
Supervisor with tool-calling	worker agents wrapped as tools of the supervisor agent	very simple; workers get only the tool input
Network / swarm	any agent can hand off to any other (Command(goto=...))	flexible, fewer hops; harder to control
Hierarchical	supervisors of teams (subgraphs)	scales to large systems; complex
Custom workflow	fixed edges between agents (pipeline)	predictable

handoffs.py

```
# Handoff tool: lets an agent transfer control to another agent node in the parent graph
from langchain_core.tools import InjectedToolCallId
from langgraph.prebuilt import InjectedState
def make_handoff(agent_name: str):
    @tool(f"transfer_to_{agent_name}", description=f"Transfer the conversation to {agent_name}.")
    def handoff(state: Annotated[dict, InjectedState], tool_call_id: Annotated[str, InjectedToolCallId]):
        msg = ToolMessage(f"Transferred to {agent_name}", tool_call_id=tool_call_id)
        return Command(goto=agent_name, graph=Command.PARENT, update={"messages": state["messages"] + [msg]})
    return handoff

billing = create_agent(model=llm, tools=[refund_status, make_handoff("tech")], name="billing")
tech = create_agent(model=llm, tools=[reset_password, make_handoff("billing")], name="tech")
g = StateGraph(MessagesState)
g.add_node("billing", billing); g.add_node("tech", tech)
g.add_conditional_edges(START, lambda s: "tech" if "password" in s["messages"][-1].content.lower() else
        "billing")
swarm = g.compile(checkpointer=InMemorySaver())
```

supervisor.py

```
class Next(BaseModel): next: Literal["researcher", "coder", "FINISH"]; reason: str
def supervisor(state: MessagesState) -> Command[Literal["researcher", "coder", "__end__"]]:
    d = llm.with_structured_output(Next).invoke([SystemMessage("Route to the best worker, or FINISH when done.")] +
        state["messages"])
    return Command(goto=END if d.next == "FINISH" else d.next)
def researcher(state: MessagesState) -> Command[Literal["supervisor"]]:
    r = research_agent.invoke(state)
    return Command(update={"messages": [AIMessage(r["messages"][-1].content, name="researcher")]}, goto="supervisor")
def coder(state: MessagesState) -> Command[Literal["supervisor"]]:
    r = code_agent.invoke(state)
    return Command(update={"messages": [AIMessage(r["messages"][-1].content, name="coder")]}, goto="supervisor")
g = StateGraph(MessagesState)
for n, f in [("supervisor", supervisor), ("researcher", researcher), ("coder", coder)]: g.add_node(n, f)
g.add_edge(START, "supervisor"); team = g.compile()
```

Prebuilt helpers exist as separate packages (langgraph-supervisor, langgraph-swarm). Design tips: give each agent a focused prompt and a small tool set; decide what history each agent sees (full vs. only its task); name messages by agent; cap iterations; trace with LangSmith.

CHAPTER 21

Functional API

@entrypoint • @task • futures & parallelism • persistence & previous • entrypoint.final • HITL with interrupt • when to use

The Functional API adds LangGraph's runtime features (checkpointing, memory, HITL, streaming) to ordinary Python functions without defining a graph.

functional_api.py

```
from langgraph.func import entrypoint, task
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt

@task                                        # unit of work; result is checkpointed → not recomputed on
       resume
def write_essay(topic: str) -> str: return llm.invoke(f"Essay on {topic}").content
@task
def grade(essay: str) -> int: return int(llm.invoke(f"Score 1-10 only: {essay}").content.strip())

@entrypoint(checkpointer=InMemorySaver())   # the workflow
def workflow(topic: str) -> dict:
    essay = write_essay(topic).result()                    # tasks return futures
    scores = [grade(essay) for _ in range(3)]              # launched in parallel
    avg = sum(s.result() for s in scores) / 3
    approved = interrupt({"essay": essay, "score": avg})    # HITL works here too
    return {"essay": essay, "score": avg, "approved": approved}

cfg = {"configurable": {"thread_id": "e1"}}
for ev in workflow.stream("LangGraph", cfg): print(ev)
workflow.invoke(Command(resume=True), cfg)
```

- **previous**: an entrypoint can declare a previous parameter to receive its last return value for that thread (short-term memory).
- **entrypoint.final(value=..., save=...)**: return one value to the caller but save a different value for next time.
- Tasks must have serialisable inputs/outputs; put side effects inside tasks so they are not repeated on resume.
- Same streaming, store and context features as the Graph API.

CHAPTER 22

Deployment, Testing & Production

LangGraph Server / Platform • langgraph.json & CLI • Studio • SDK & threads/runs API • self-hosting with FastAPI • testing graphs • evaluation • performance & reliability checklist

22.1 Deployment options

Option	What you get
LangGraph Server / Platform (cloud, hybrid, self-hosted)	HTTP API for assistants, threads, runs, cron jobs, streaming, background runs, task queues, built-in persistence and store, double-texting handling
langgraph CLI: langgraph dev	local dev server + LangGraph Studio (visual graph debugger: run, inspect state, edit, time-travel)
langgraph build / up	Docker image of your graph server
Self-host in FastAPI	your own endpoints calling graph.ainvoke/astream with a Postgres checkpointer
LangGraph SDK (langgraph_sdk)	Python/JS client: client.threads.create(), client.runs.stream(thread_id, assistant_id, input=...)

langgraph.json

```
{
  "dependencies": ["."],
  "graphs": { "support_agent": "./app/graph.py:graph" },
  "env": ".env",
  "python_version": "3.11"
}
```

cli_and_sdk

```
pip install -U "langgraph-cli[inmem]"
langgraph dev                       # starts API on http://127.0.0.1:2024 and opens Studio
# client side
from langgraph_sdk import get_client
client = get_client(url="http://127.0.0.1:2024")
thread = await client.threads.create()
async for chunk in client.runs.stream(thread["thread_id"], "support_agent", input={"messages": [{"role": "user", "content":
        "hi"}]}, stream_mode="updates"):
    print(chunk.event, chunk.data)
```

Self-hosted FastAPI pattern

fastapi_langgraph.py

```
from contextlib import asynccontextmanager
from langgraph.checkpoint.postgres.aio import AsyncPostgresSaver
@asynccontextmanager
async def lifespan(app):
    async with AsyncPostgresSaver.from_conn_string(DB_URI) as saver:
        await saver.setup(); app.state.graph = builder.compile(checkpointer=saver); yield
app = FastAPI(lifespan=lifespan)

@app.post("/chat/{thread_id}")
async def chat(thread_id: str, body: ChatIn):
    cfg = {"configurable": {"thread_id": thread_id}}
    async def gen():
        async for tok, meta in app.state.graph.astream({"messages": [("user", body.message)]}, cfg, stream_mode="messages"):
            if tok.content and meta["langgraph_node"] == "agent": yield f"data: {tok.content}\n\n"
    return StreamingResponse(gen(), media_type="text/event-stream")

@app.post("/resume/{thread_id}")
async def resume(thread_id: str, decision: dict):
    return await app.state.graph.ainvoke(Command(resume=decision), {"configurable": {"thread_id": thread_id}})
```

22.2 Testing LangGraph applications

test_graph.py

```
import pytest
from langchain_core.language_models.fake_chat_models import GenericFakeChatModel

def test_router_goes_to_billing(monkeypatch):
    monkeypatch.setattr(nodes, "router_llm", FakeRouter(step="billing"))      # mock the LLM dependency
    out = graph.invoke({"input": "I was charged twice"})
    assert out["decision"] == "billing"

def test_single_node():
    assert nodes.count({"text": "a b c"}) == {"word_count": 3}                 # nodes are plain functions → unit-test
        directly

def test_partial_execution():
    g = builder.compile(checkpointer=InMemorySaver()); cfg = {"configurable": {"thread_id": "t"}}
    g.update_state(cfg, {"docs": [Document(page_content="x")]}, as_node="retrieve")   # start mid-graph
    out = g.invoke(None, cfg, interrupt_after=["grade_docs"])                    # run just one segment
    assert g.get_state(cfg).values["docs"]
```

Level	What to test
Unit	each node function, router functions, tools, parsers (fake LLMs)
Graph paths	each branch reachable; loops terminate; recursion limit handling
Trajectory evals	expected sequence of tool calls (agentevals / LangSmith)
End-to-end	golden dataset + LLM-as-judge for final answers
Load	concurrent threads, latency p95, checkpointer DB load

22.3 Production checklist

- Persistent checkpointer (Postgres) + store; unique thread_id per conversation; retention/cleanup policy.
- Typed, minimal state; store IDs instead of huge blobs; explicit reducers for parallel keys.
- Recursion limits + retry counters in state; RetryPolicy on flaky nodes; ToolNode error handling; fallbacks.
- Idempotent nodes; side effects after approval; HITL interrupts for risky actions.
- Streaming (messages + updates) for UX; async everywhere in servers.
- Context engineering: trimming/summarisation, focused prompts per node, small tool sets per agent.
- Observability: LangSmith tracing with tags/metadata (user, version); cost and latency dashboards.

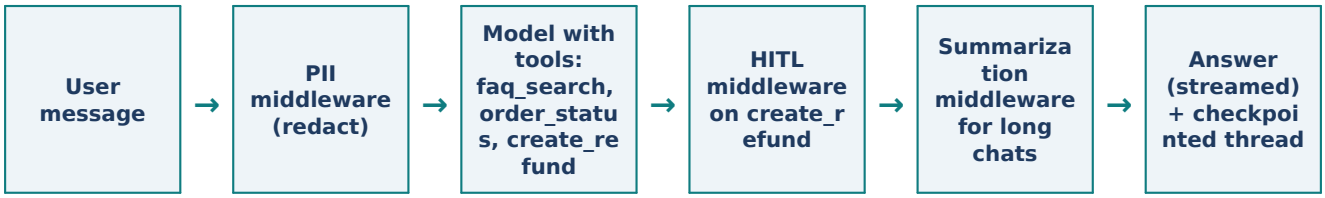
- Security: least-privilege tools, prompt-injection defences, PII handling, auth on endpoints.
- Version graphs and prompts; evaluate in CI before deploying.

CHAPTER 23

End-to-End Projects

Project 1: customer-support agent (LangChain create_agent + middleware) • Project 2: multi-step research assistant (LangGraph) • file structure • how the flow works

23.1 Project 1 — Customer support agent with create_agent



structure

```
support_agent/
├── main.py          # FastAPI: /chat (SSE stream), /resume, /health
├── agent.py         # create_agent(...) with middleware + checkpointer
├── tools.py         # @tool faq_search, order_status, create_refund
├── rag.py           # loaders → splitter → Chroma → hybrid retriever
├── prompts.py       # SYSTEM_PROMPT
├── schemas.py       # Pydantic: ChatIn, Ctx, FinalAnswer
└── tests/           # test_tools.py, test_agent.py (fake model)
```

agent.py / tools.py

```
# tools.py
@tool
def faq_search(query: str) -> str:
    """Search FAQs and policies. Use for any policy/how-to question."""
    return format_docs(retriever.invoke(query))
@tool
def order_status(order_id: str, runtime: ToolRuntime) -> str:
    """Get status of the CURRENT user's order by ID like ORD-123."""
    return orders.status(order_id, user=runtime.context.user_id)
@tool
def create_refund(order_id: str, amount: float, reason: str) -> str:
    """Create a refund. Requires human approval."""
    return payments.refund(order_id, amount, reason)

# agent.py
agent = create_agent(
    model="google_genai:gemini-2.5-flash", tools=[faq_search, order_status, create_refund],
    system_prompt=SYSTEM_PROMPT, context_schema=Ctx, checkpointer=saver,
    middleware=[PIIMiddleware("email", strategy="redact", apply_to_input=True),
                SummarizationMiddleware(model="google_genai:gemini-2.5-flash", trigger={"tokens": 4000}),
                ModelCallLimitMiddleware(run_limit=8),
                HumanInTheLoopMiddleware(interrupt_on={"create_refund": {"allowed_decisions": ["approve", "reject"]}})])
```

23.2 Project 2 — Research assistant with LangGraph



research_graph.py

```
class ResearchState(TypedDict):
    topic: str; questions: list[str]; notes: Annotated[list[str], operator.add]
    report: str; verdict: str; feedback: str; rounds: int
class Plan(BaseModel): questions: list[str] = Field(max_length=5)
class Review(BaseModel): verdict: Literal["good", "revise"]; feedback: str

def plan(s):
    hint = f" Address feedback: {s['feedback']}" if s.get("feedback") else ""
    return {"questions": llm.with_structured_output(Plan).invoke(f"Sub-questions to research
        {s['topic']}.{hint}").questions}
def fan_out(s): return [Send("search_and_note", {"q": q}) for q in s["questions"]]
def search_and_note(s: dict):
    writer = get_stream_writer(); writer({"status": f"searching: {s['q']}"})
    hits = web_search.invoke(s["q"])
    return {"notes": [llm.invoke(f"Note key facts with URLs for '{s['q']}':\n{hits}").content]}
def write_report(s): return {"report": llm.invoke(f"Write a cited report on {s['topic']} from notes:\n" +
        "\n".join(s["notes"])).content,
                             "rounds": s.get("rounds", 0) + 1}
def review(s):
    r = llm.with_structured_output(Review).invoke(f"Review this report for gaps and unsupported claims:\n{s['report']}")
    return {"verdict": r.verdict, "feedback": r.feedback}
def after_review(s): return "human_approval" if s["verdict"] == "good" or s["rounds"] >= 2 else "plan"
def human_approval(s) -> Command[Literal["__end__", "plan"]]:
    ok = interrupt({"report": s["report"]})
    return Command(goto=END) if ok else Command(goto="plan", update={"feedback": "Human requested changes"})

b = StateGraph(ResearchState)
for n, f in [("plan", plan), ("search_and_note", search_and_note), ("write_report", write_report), ("review", review),
        ("human_approval", human_approval)]:
    b.add_node(n, f)
b.add_edge(START, "plan"); b.add_conditional_edges("plan", fan_out, ["search_and_note"])
b.add_edge("search_and_note", "write_report"); b.add_edge("write_report", "review")
b.add_conditional_edges("review", after_review, ["human_approval", "plan"])
graph = b.compile(checkpointer=InMemorySaver())
```

How the flow works, step by step

1. invoke({'topic': ...}, cfg) → super-step 1 runs **plan**, writing questions.
2. The conditional edge returns one Send per question → super-step 2 runs **search_and_note** N times in parallel; operator.add merges notes.
3. **write_report** runs after all parallel branches finish; then **review** grades it.
4. after_review loops back to plan (max 2 rounds) or goes to **human_approval**, which interrupts and persists.
5. The client shows the report; invoke(Command(resume=True), cfg) re-runs human_approval, interrupt() returns True, and the graph ends.

CHAPTER 24

Interview Questions & Answers

LangChain core • LCEL & Runnables • RAG components • tools & agents • LangGraph state & execution • persistence, memory & HITL • streaming • multi-agent • production • code-reading MCQs

24.1 LangChain fundamentals

Q1. What is LangChain?

An open-source framework with standard, provider-agnostic interfaces for building LLM applications — models, prompts, parsers, tools, retrievers — composable through the Runnable interface, plus a prebuilt agent (create_agent) with middleware.

Q2. What is langchain-core vs langchain vs langchain-community vs langchain-classic?

core holds base abstractions and LCEL; langchain holds high-level agent APIs (create_agent, middleware, init_chat_model); community holds third-party integrations; classic holds legacy chains, AgentExecutor and old memory classes moved out in v1.

Q3. What changed in LangChain v1?

create_agent became the standard agent API (replacing LangGraph's create_react_agent), middleware was introduced for customisation, standard content blocks unified provider outputs, and legacy features moved to langchain-classic.

Q4. Chat model vs LLM class?

Chat models take a list of role-tagged messages and return an AIMessage (supporting tools, multimodal content); legacy LLM classes take and return plain strings.

Q5. What message types exist?

SystemMessage, HumanMessage, AIMessage (with tool_calls and usage metadata), ToolMessage (tool result with tool_call_id) and AIMessageChunk for streaming; RemoveMessage is used in LangGraph.

Q6. How do you switch providers with minimal code change?

Use init_chat_model with a 'provider:model' string or configurable_alternatives; the rest of the chain uses the shared chat-model interface.

Q7. Why use ChatPromptTemplate instead of f-strings?

Templates are Runnables that validate variables, support message roles, MessagesPlaceholder for history, partials and few-shot examples, and compose and trace inside chains.

Q8. What is MessagesPlaceholder?

A slot in a chat prompt where a variable-length list of messages (chat history or agent scratchpad) is inserted.

Q9. How do you include literal braces in a prompt?

Escape them as {{ and }} — otherwise they are parsed as variables.

Q10. with_structured_output vs output parsers?

with_structured_output uses the provider's native tool calling/JSON schema to return validated objects and is more reliable; parsers add format instructions and parse free text, useful when native support is missing.

24.2 Runnables and LCEL

Q11. What is a Runnable?

The common interface implemented by almost every LangChain component, providing invoke, batch, stream, their async versions and astream_events, plus wrappers like with_retry and with_fallbacks.

Q12. What is LCEL?

LangChain Expression Language — composing runnables declaratively with the | operator and dicts, giving automatic streaming, batching/parallelism, async, retries and tracing.

Q13. What does the | operator create?

A RunnableSequence in which each step's output becomes the next step's input.

Q14. What happens when you put a dict in a chain?

It is coerced into a RunnableParallel: every value runs on the same input concurrently and the output is a dict with the same keys.

Q15. RunnablePassthrough vs RunnablePassthrough.assign?

Passthrough returns the input unchanged; assign keeps all input keys and adds new keys computed from the input.

Q16. How do you put a custom Python function into a chain?

Wrap it in RunnableLambda (or just use it after a | where coercion applies) or decorate it with @chain.

Q17. How do you implement routing in LCEL?

A RunnableLambda that returns the chosen runnable based on the input (often after a structured-output classifier), or RunnableBranch with (condition, runnable) pairs and a default.

Q18. invoke vs batch vs stream?

invoke processes one input; batch processes a list concurrently (max_concurrency controls threads); stream yields chunks as they are produced.

Q19. Why might stream() not stream token by token?

A step that needs the whole input (e.g. a non-generator RunnableLambda or a parser that cannot parse partial input) blocks streaming until it receives everything.

Q20. What is RunnableConfig used for?

Passing tags, metadata, callbacks, run_name, max_concurrency, recursion_limit and configurable values (like thread_id) that propagate to all child runnables.

Q21. bind vs bind_tools?

bind fixes invocation kwargs such as stop sequences; bind_tools is a specialised bind that converts tools into the provider's tool schema and attaches them to the model.

Q22. with_retry vs with_fallbacks?

with_retry re-runs the same runnable on errors with backoff; with_fallbacks tries different runnables (e.g. another model) if the primary fails.

Q23. What are configurable_fields and configurable_alternatives?

Ways to change a runnable's parameters (e.g. temperature) or swap whole components (e.g. Gemini vs GPT) at runtime via config['configurable'].

24.3 Chains, memory and RAG components

Q24. What replaced LLMChain?

The LCEL expression prompt | llm | StrOutputParser(); LLMChain lives in langchain-classic.

Q25. Stuff vs map-reduce vs refine?

Stuff puts all documents in one prompt; map-reduce processes documents separately then combines; refine updates an answer document by document sequentially.

Q26. What do create_stuff_documents_chain, create_retrieval_chain and create_history_aware_retriever do?

The first formats documents into the prompt's {context}; the second retrieves documents and passes them to a document chain, returning answer and context; the third rewrites a follow-up question into a standalone query using chat history before retrieval.

Q27. How is memory implemented in modern LangChain?

Either RunnableWithMessageHistory with a chat message history per session_id, or — preferred for agents — a LangGraph checkpointer keyed by thread_id plus a Store for long-term memory.

Q28. ConversationBufferMemory vs ConversationSummaryBufferMemory?

Buffer keeps every message (grows unbounded); SummaryBuffer keeps recent messages verbatim and summarises older ones to stay within a token limit.

Q29. What is a retriever vs a vector store?

A vector store stores embeddings and supports similarity search; a retriever is a Runnable interface mapping a query string to documents — it can wrap a vector store (as_retriever) or BM25, web search, etc.

Q30. Explain search_type options of as_retriever.

similarity returns top-k; mmr returns relevant but diverse results (fetch_k candidates, lambda_mult balance); similarity_score_threshold returns only documents above a score.

Q31. How do you build hybrid search in LangChain?

EnsembleRetriever combining BM25Retriever and a vector retriever with weights, which fuses rankings with weighted Reciprocal Rank Fusion; optionally add a ContextualCompressionRetriever with a cross-encoder reranker.

Q32. What does ParentDocumentRetriever do?

Indexes small child chunks for precise matching but returns their larger parent chunks from a docstore for fuller context.

Q33. How do you avoid duplicates when re-ingesting documents?

Use stable IDs with upserts or the indexing API with a RecordManager, which hashes documents and cleans up changed or deleted sources.

24.4 Tools and agents

Q34. How does tool calling work?

Tool schemas are bound to the model; the model returns an AIMessage with tool_calls (name, args, id); the application executes the tools and sends ToolMessages with matching tool_call_id; the model then continues.

Q35. Does the LLM execute the tool?

No — it only produces a structured request; your code, ToolNode or create_agent executes it.

Q36. How is a tool's description created with @tool?

From the function name, docstring and type hints (or an explicit args_schema); the docstring tells the model when to use the tool.

Q37. What is InjectedToolArg / ToolRuntime?

Mechanisms to supply arguments at runtime that the model must not generate — like user ID, state, store or context — keeping them hidden from the tool schema.

Q38. What is response_format='content_and_artifact'?

The tool returns a short content string for the model plus a raw artifact (e.g. full rows or a dataframe) attached to the ToolMessage for application use.

Q39. AgentExecutor vs create_agent?

AgentExecutor is the legacy agent runtime (langchain-classic); create_agent is the v1 standard built on LangGraph with persistence, streaming and middleware.

Q40. What is middleware in LangChain v1?

Composable hooks around the agent loop — before/after agent, before/after model, wrap model call, wrap tool call, dynamic prompt — used for summarisation, PII redaction, HITL approval, call limits, fallbacks and dynamic model or tool selection.

Q41. How do you get a structured final answer from an agent?

Pass response_format with a Pydantic schema (ToolStrategy or ProviderStrategy); the result includes structured_response.

Q42. How do you add memory to create_agent?

Pass a checkpointer and invoke with config {'configurable': {'thread_id': ...}}; pass a store for long-term memory.

Q43. Which agent pattern relies heavily on tool integration?

ReAct — reasoning interleaved with tool actions and observations.

24.5 LangGraph core

Q44. What is LangGraph and why use it over LCEL?

A low-level orchestration runtime for stateful graphs; unlike LCEL chains it supports cycles, branching, shared state, persistence, human-in-the-loop, streaming and multi-agent subgraphs.

Q45. Explain state, nodes and edges.

State is the shared typed data; nodes are functions that read state and return partial updates; edges define which node runs next, either fixed or conditional.

Q46. What is a super-step?

One execution tick in which all scheduled nodes run in parallel on the same state snapshot, after which their updates are applied via reducers and a checkpoint is saved.

Q47. What is a reducer and when is it required?

A function defining how updates merge into a key (overwrite by default, operator.add, add_messages, custom). It is required when multiple nodes may write the same key in the same super-step.

Q48. What does add_messages do?

Appends messages, converts dict/tuple formats to message objects, replaces messages with the same id, and supports deletion via RemoveMessage.

Q49. What is MessagesState?

A prebuilt state TypedDict with a messages key using the add_messages reducer; extend it to add fields.

Q50. Conditional edge vs Command?

A conditional edge uses a separate router function on state to pick the next node; Command is returned by a node to both update state and specify goto, useful for handoffs.

Q51. What is Send used for?

Dynamic fan-out (map-reduce): a conditional edge returns multiple Send(node, state) objects to run a node in parallel with different inputs determined at runtime.

Q52. How do you fan-in after parallel nodes?

add_edge([a, b, c], 'combine') waits for all; give parallel-written keys reducers; use defer=True for branches of unequal length.

Q53. What is the recursion limit?

The maximum number of super-steps per run (default 25); exceeding it raises GraphRecursionError — set it in config or add your own loop counters.

Q54. Why must nodes return partial updates instead of mutating state?

The runtime applies updates through reducers and checkpoints them; in-place mutation bypasses this, breaking parallel merges, persistence and time travel.

Q55. What are input/output schemas?

Optional schemas that restrict which keys a graph accepts as input and returns as output while the internal state can hold more keys.

Q56. Graph API vs Functional API?

The Graph API defines explicit nodes and edges with shared state; the Functional API uses @entrypoint and @task with normal Python control flow while still providing persistence, streaming and interrupts.

24.6 Persistence, memory, HITL and streaming

Q57. What does a checkpointer do?

Saves a snapshot of graph state at every super-step, grouped by thread_id, enabling conversation memory, interrupts/HITL, time travel and fault-tolerant resumption.

Q58. What is a thread?

A sequence of checkpoints for one conversation or run, identified by config['configurable']['thread_id'].

Q59. InMemorySaver vs PostgresSaver?

InMemorySaver keeps checkpoints in RAM (lost on restart, for dev/tests); PostgresSaver persists them in a database for production.

Q60. What does get_state return?

A StateSnapshot with values, next nodes, config (with checkpoint_id), metadata and pending tasks/interrupts.

Q61. How do you time travel?

Find a checkpoint via get_state_history, then invoke(None, that_config) to replay, or update_state on it to fork and continue on a new branch.

Q62. Short-term vs long-term memory in LangGraph?

Short-term is thread-scoped state persisted by the checkpointer; long-term is cross-thread data in a Store organised by namespaces and keys, optionally with semantic search.

Q63. How do you handle very long conversations?

Trim messages before model calls, delete old messages with RemoveMessage, or summarise older messages into a summary key (or SummarizationMiddleware in create_agent).

Q64. How does interrupt() work?

It pauses execution inside a node, saves state via the checkpointer and returns the payload to the caller under __interrupt__; resuming with Command(resume=value) re-runs the node from the start and interrupt() returns value.

Q65. What are pitfalls of interrupts?

Code before interrupt() re-executes on resume (must be idempotent), interrupt() must not be swallowed by try/except, multiple interrupts are matched by order, and a checkpointer plus thread_id are required.

Q66. Static vs dynamic interrupts?

Static interrupts (interrupt_before/after at compile or run time) pause at node boundaries, mainly for debugging; dynamic interrupt() pauses conditionally inside node logic with a payload.

Q67. What stream modes exist?

values (full state), updates (per-node deltas), messages (LLM tokens with metadata), custom (via get_stream_writer), and debug; several can be combined.

Q68. How do you stream only the tokens from one node?

Use stream_mode='messages' and filter by metadata['langgraph_node'] (or tags).

Q69. What does the token streaming give users?

Lower perceived latency — users see output immediately — without changing accuracy.

24.7 Multi-agent and production

Q70. Supervisor vs swarm architecture?

A supervisor centrally routes tasks to worker agents that report back; in a swarm agents hand off directly to each other via handoff tools that return Command(goto=...).

Q71. How does an agent in a subgraph hand off to another agent in the parent graph?

By returning Command(goto='other_agent', graph=Command.PARENT, update=...) — typically from a handoff tool.

Q72. When should you use subgraphs?

To modularise complex systems (each agent or team as its own graph), reuse components, and give parts private state; add them directly if keys are shared or call them inside a node with state mapping otherwise.

Q73. How do you deploy a LangGraph app?

Via LangGraph Platform/Server using langgraph.json and the CLI (with Studio for debugging), or self-host inside FastAPI with a Postgres checkpointer and SSE streaming.

Q74. How do you test a LangGraph graph?

Unit-test nodes and routers as plain functions with fake LLMs, test each path and loop termination, run partial execution using update_state with as_node, and evaluate trajectories and final answers on golden datasets.

Q75. How do you make a graph fault tolerant?

Persistent checkpointer, RetryPolicy on flaky nodes, tool error handling, fallbacks, idempotent nodes, recursion limits and resuming threads with invoke(None, cfg).

Q76. How do you control cost in agents?

Cheaper models for routing and grading, call-limit middleware or counters, trimming/summarisation, caching, small tool sets, and LangSmith cost tracking.

24.8 Code-reading MCQs (assessment style)

Q77. Output of *(RunnablePassthrough.assign(n=lambda x: len(x['t']))).invoke({'t': 'abc'})*?

{'t': 'abc', 'n': 3} — assign keeps existing keys and adds n.

Q78. Two parallel nodes return {'items': ['a']} and {'items': ['b']} and items has no reducer. Result?

InvalidUpdateError — concurrent writes to a key without a reducer. Fix with Annotated[list, operator.add].

Q79. A node returns {'messages': [AIMessage('hi', id='42')]} and a message with id '42' already exists (add_messages). Result?

The existing message is replaced, not duplicated.

Q80. Graph compiled without a checkpointer calls interrupt(). What happens?

It fails — interrupts require a checkpointer (and a thread_id) to persist and resume.

Q81. What does tools_condition return when the last AIMessage has no tool_calls?

END ('__end__'), so the graph finishes.

Q82. In *if tool_error: response = fallback_llm_answer(user_query)*, what pattern is shown?

Fallback / graceful error handling — answering with the LLM when a tool fails.

Q83. Role of conversation_state in a support agent snippet?

Session-scoped memory of the conversation used for context-aware replies — not permanent storage across all sessions.

Q84. Which call resumes a paused graph with human input?

graph.invoke(Command(resume=value), config_with_same_thread_id).

Q85. Which stream_mode gives token-by-token LLM output?

'messages'.

Q86. What is printed by chain.invoke when chain = prompt | llm (no parser)?

An AIMessage object, not a string — add StrOutputParser() or read .content.

CHAPTER 25

Cheat Sheets

Imports • LCEL • create_agent • LangGraph skeletons • common errors and fixes • legacy → modern mapping

25.1 Import cheat sheet (v1)

What	Import
chat model factory	from langchain.chat_models import init_chat_model
messages	from langchain_core.messages import SystemMessage, HumanMessage, AIMessage, ToolMessage, RemoveMessage, trim_messages
prompts	from langchain_core.prompts import ChatPromptTemplate, PromptTemplate, MessagesPlaceholder, FewShotChatMessagePromptTemplate
parsers	from langchain_core.output_parsers import StrOutputParser, JsonOutputParser, PydanticOutputParser
runnables	from langchain_core.runnables import RunnablePassthrough, RunnableParallel, RunnableLambda, RunnableBranch, RunnableConfig, chain
tools	from langchain.tools import tool, ToolRuntime (or langchain_core.tools)
agent	from langchain.agents import create_agent
middleware	from langchain.agents.middleware import SummarizationMiddleware, HumanInTheLoopMiddleware, PIIMiddleware, before_model, after_model, wrap_model_call, wrap_tool_call, dynamic_prompt
documents / splitters	from langchain_core.documents import Document; from langchain_text_splitters import RecursiveCharacterTextSplitter
vector store	from langchain_chroma import Chroma; from langchain_community.vectorstores import FAISS
legacy	from langchain_classic.chains import ...; from langchain_classic.agents import AgentExecutor
graph	from langgraph.graph import StateGraph, MessagesState, START, END; from langgraph.graph.message import add_messages
types	from langgraph.types import Command, Send, interrupt, RetryPolicy, CachePolicy
prebuilt	from langgraph.prebuilt import ToolNode, tools_condition, InjectedState
persistence	from langgraph.checkpoint.memory import InMemorySaver; from langgraph.store.memory import InMemoryStore
runtime / streaming	from langgraph.runtime import Runtime; from langgraph.config import get_stream_writer
functional API	from langgraph.func import entrypoint, task

25.2 Skeletons to memorise

skeletons.py

```
# LCEL
chain = prompt | llm | StrOutputParser()
rag = {"context": retriever | format_docs, "question": RunnablePassthrough()} | prompt | llm | StrOutputParser()

# Agent (LangChain v1)
agent = create_agent(model="provider:model", tools=[...], system_prompt="...", checkpointer=InMemorySaver(),
       middleware=[...])
agent.invoke({"messages": [("user", "...")]}, {"configurable": {"thread_id": "1"}})

# LangGraph
class State(MessagesState): extra: str
b = StateGraph(State)
b.add_node("agent", call_model); b.add_node("tools", ToolNode(tools))
b.add_edge(START, "agent"); b.add_conditional_edges("agent", tools_condition); b.add_edge("tools", "agent")
graph = b.compile(checkpointer=InMemorySaver())
for mode, chunk in graph.stream({"messages": [("user", "hi")]}, cfg, stream_mode=["updates", "messages"]): ...
```

25.3 Common errors and fixes

Error / symptom	Cause	Fix
KeyError / missing variables in prompt	unescaped { } in template or missing input key	escape {{ }}; check prompt.input_variables
Output is AIMessage, expected str	no parser	add StrOutputParser() or use .content
ToolMessage must follow tool_calls / 400 from provider	message ordering broken (tool result without matching tool_call_id, or trimmed history split a tool call)	keep AIMessage + its ToolMessages together; trim with start_on='human'
InvalidUpdateError: can receive only one value per step	parallel writes to a key without reducer	Annotated[..., operator.add] or other reducer
GraphRecursionError	loop never reaches END	add counters/termination; raise recursion_limit only if legitimate
Checkpointer requires thread_id	missing configurable.thread_id	pass config={'configurable': {'thread_id': ...}}
Interrupt resumes with wrong values / duplicate side effects	node re-runs from start on resume	make pre-interrupt code idempotent; move side effects after interrupt
Memory not working across invokes	no checkpointer or different thread_id each call	compile with checkpointer; reuse thread_id
ImportError for LLMChain / AgentExecutor / ConversationBufferMemory	moved in v1	pip install langchain-classic; import from langchain_classic
Streaming prints everything at the end	a blocking step in the chain	use generator functions; stream_mode='messages' in graphs
Structured output fails validation	schema too complex / model weak	simplify schema, add Field descriptions, include_raw=True, retry, stronger model

25.4 Legacy → modern mapping

Legacy	Modern
LLMChain	prompt | llm | parser
SequentialChain	RunnablePassthrough.assign chains / LangGraph
ConversationChain + Memory	RunnableWithMessageHistory / checkpointer
RetrievalQA	LCEL RAG / create_retrieval_chain / agentic RAG
initialize_agent / AgentExecutor	create_agent
langgraph.prebuilt.create_react_agent	langchain.agents.create_agent
Agent callbacks for guardrails	middleware
LangServe	FastAPI or LangGraph Server
.run() / .predict() / __call__	.invoke()

Sources

[1] LangChain blog — *LangChain and LangGraph Agent Frameworks Reach v1.0 Milestones* (Oct 22, 2025), langchain.com/blog/langchain-langgraph-1dot0

[2] LangChain docs — *What's new in LangChain v1*, docs.langchain.com/oss/python/releases/langchain-v1

[3] LangChain docs — *What's new in LangGraph v1*, docs.langchain.com/oss/python/releases/langgraph-v1

[4] LangChain reference — *Middleware*, reference.langchain.com/python/langchain/middleware

[5] LangChain blog — *LangChain & LangGraph 1.0 alpha releases* (Sep 2, 2025)

APIs evolve quickly; verify exact signatures against the current documentation for your installed versions.